Notice·Board

Complete Project Handbook

A ROLE-BASED COLLEGE ONLINE NOTICE BOARD SYSTEM

Every page, component, class, table, endpoint and rule in the project — explained from first principles for a reader who has never seen this codebase before. Includes 80+ viva questions with answers, a slide-by-slide presentation script, and a full demo walkthrough.

Table of Contents

1	Project Overview	3
2	Problem Statement	5
3	Objectives	6
4	Complete Feature List	7
5	Technologies Used — what, why and where	10
6	Complete Architecture	14
7	Complete Folder Structure	16
8	Frontend — pages, components, routing, state, validation	18
9	Backend — packages, classes and the request journey	23
10	Database — every table, every column, every relationship	28
11	API Documentation — all 73 endpoints	33
12	Authentication — login to protected access	40
13	Application Workflow — what happens on startup and after	42
14	Business Logic — all major workflows	44
15	Security Features	48
16	Error Handling	50
17	Validation	51
18	Challenges Faced During Development	52
19	Possible Improvements	55
20	Future Scope	56
21	Expected Viva Questions — 87 questions with answers	57
22	Difficult Faculty Questions	75
23	Complete Presentation Script	77
24	Complete Demo Script	80
25	Quick Revision Notes — 15-minute refresher	83

§1 Project Overview

The College Online Notice Board System is a full-stack web application that replaces the physical notice board. Authorised staff create, schedule, approve and publish notices; **each notice is visible only to the audience its creator selected**; and visitors who are not signed in see a public website with admissions information and public notices.

1.1 What the system is, in one paragraph

The application is built as three tiers. A React single-page application runs in the user's browser and shows one of two faces: a public website for visitors, or a role-based portal for people who have signed in. It talks over HTTP, using JSON and a Bearer token, to a Spring Boot REST API that holds every rule: who may address whom, who may approve what, and who may see which notice. The API stores everything in a MySQL database and delegates three specific jobs to outside services: **Cloudinary** for attachment storage, **Gmail SMTP** for email (optional), and the **Anthropic API** for the AI drafting assistant (optional, with an offline fallback).

1.2 The five roles

Role	How the account is created	What the role can do
Super Admin	Seeded on first start in development; later created by another Super Admin.	Sees and publishes everything. Manages users, departments, categories and the audit log. Can suspend or close accounts. Can delete any notice.
Principal	Created by the Super Admin (cannot self-register).	Publishes notices to any audience without approval. Approves HOD account requests. Manages class teachers in any department. Publishes public notices for either wing.
HOD	Self-registers; the **Principal** approves. Only one HOD per department.	Creates notices for their own department and publishes them directly. Approves or rejects faculty notices. Approves teacher and student accounts of the department. Assigns class teachers.
Faculty (shown as "Teacher")	Self-registers; the **department HOD** approves.	Writes notices for students of their own department. Every notice goes to the HOD for approval. As class teacher, approves student registrations for that class.
Student	Self-registers; the **class teacher** approves (falls back to HOD, then Principal).	Reads notices addressed to them, acknowledges important ones, saves notices to a reading list, receives in-app and email notifications. Cannot create notices.

1.3 The single most important design idea

> **Visibility is decided on the server, by one rule, written twice and tested to agree**
>
> In most college notice-board projects, "who can see this?" is answered by the React page hiding things. That is not security: anyone can call the API directly. In this project every notice has one or more `NoticeTarget` rows. **A row matches a user when every non-null field in the row equals the user's value** (role, department, semester, class, user). A notice is visible to a user if **any** row matches.
>
> That rule is implemented once in Java (`TargetMatcher`) and once as a SQL specification (`NoticeSpecifications.targetedTo`) so that list pages and counts are filtered inside the database. Both are covered by tests. Opening `/api/notices/42` for a notice you are not addressed by returns **403 "You are not authorized to view this notice"**, so changing the ID in the URL does not help.

1.4 Project size at a glance

Metric	Backend	Frontend
Lines of source code	≈ 7,256 lines of Java (main) + ≈ 1,429 lines of tests	≈ 3,560 lines of JS/JSX + ≈ 453 lines of CSS
Primary units	13 controllers · 28 service classes and interfaces · 12 repositories · 12 entities + 9 enums · 55 DTO records in 18 files · 11 exception classes · 13 config classes	19 page files serving 29 routes · 18 components · 3 contexts · 2 hook files · 12 API modules · 2 utility modules
Database	12 tables in the `college_notice_board` schema, created automatically by Hibernate on first run	
REST endpoints	73 endpoints across 13 controllers (12 of them public, no sign-in needed)	
Automated tests	72 test methods in 8 test classes (unit tests, Mockito tests, and Spring Boot integration tests on an in-memory H2 database)	

§2 Problem Statement

A physical notice board is one piece of paper pinned to one wall. That single fact creates a chain of problems for everyone who writes, approves or reads a notice.

2.1 From the student's point of view

- **Reach depends on location.** A notice is only seen by whoever walks past the board, so absent students miss it. There is no way to receive it remotely.
- **No relevance filter.** A semester-6 exam timetable and a mechanical-department workshop sit side by side. The student must read everything to find what applies to them.
- **No confirmation.** Nothing records that a student saw an urgent circular, and nothing reminds them.
- **Notices disappear.** Once taken down, the notice is gone. Timetables and forms that were pinned cannot be taken away or looked up later.
- **Visitors and applicants are left out.** Parents and prospective students must visit the campus to learn about admissions.

2.2 From the faculty and HOD point of view

- **No way to address only their own class.** A teacher who wants to tell semester 6, class A about a lab submission has to tell the class in person or pin a sheet in front of everyone.
- **Approval is verbal.** There is no trace of who approved a faculty notice, or why one was turned down.
- **No scheduling or expiry.** Someone must remember to put a notice up on the right day and to take it down after the deadline.

2.3 From the administration's point of view

- **No control over who may post to whom.** Anyone with a pin can address the whole college.
- **No accountability.** There is no history of who created, approved, published or removed a notice.
- **No data.** The Principal cannot tell how many of the intended recipients actually read an urgent notice.
- **Onboarding is manual.** New students, teachers and HODs are known only by paper registers; there is no verified, role-based account list.

> **The problem, stated formally**
> Students, teachers, HODs and the Principal each hold one fragment of the communication picture, and no shared, authoritative record exists. The result is unreliable reach, no relevance filtering, no approval trail and no read tracking. The problem addressed is therefore: **build a single verified platform where a notice reaches exactly the audience its author chose, is approved by the right person first, and leaves an auditable record of who saw it.**

§3 Objectives

#	Objective	How it is met in the implementation
1	One platform serving five roles	One React app and one API. Role-based routing on the client (`ProtectedRoute`) and role-based authorisation on the server (Spring Security filter chain plus `@PreAuthorize`).
2	Show every notice only to its intended audience	`NoticeTarget` rows, `TargetMatcher` (Java) and `NoticeSpecifications.targetedTo` (SQL), and `NoticeAccessService` checked on every read path: list, detail, attachments, statistics.
3	Control who may address whom	`TargetPolicy` validates and authorises every target per role. A faculty member can only address students of their own department; an HOD only their own department; the Principal anyone.
4	Add quality control through approval	Faculty notices move `DRAFT → PENDING_APPROVAL` and are published only when the department HOD approves. A rejection needs a reason, and an edited notice can be resubmitted.
5	Admit only verified people	Self-registration creates a `PENDING` account that cannot sign in. The system routes it to the right approver: class teacher, HOD or Principal. The applicant never chooses the approver.
6	Publish and expire on time	A Spring `@Scheduled` job runs every minute: `SCHEDULED → PUBLISHED` when the publish time arrives, and `PUBLISHED → EXPIRED` when the expiry passes.
7	Make sure important notices are read	In-app notifications for exactly the recipients, email for Important and Urgent notices, read tracking, explicit acknowledgements, and statistics counted over intended recipients only.
8	Keep attachments safe	Cloudinary **authenticated** delivery plus 5-minute signed links issued only after the visibility check. Uploads are validated by extension, MIME type, size and file signature.
9	Serve visitors without exposing anything private	A public site backed by `/api/public/**`, which can only ever return notices that are public, published, live and not expired, through DTOs that carry no internal fields.
10	Be accountable	Every significant action writes an `AuditLog` row. Each notice has a lifecycle timeline built from that log.
11	Help authors get it right	AI draft assistant (always a draft), live recipient count while choosing the audience, preview before publishing, browser auto-save of unsaved work, drag-and-drop uploads with per-file progress.
12	Serve two wings of one college	Departments belong to Engineering or Polytechnic, each with its own programme and number of semesters. Approvals and public notices are scoped to the wing.

§4 Complete Feature List

Every feature below is present in the code. Features are grouped by the module that owns them.

4.1 Authentication and account management

- Self-registration for **students, teachers (FACULTY) and HODs**. Principal and Super Admin accounts are created by the Super Admin, never by self-registration.
- BCrypt-hashed passwords with a strict rule: 8–64 characters containing an uppercase letter, a lowercase letter, a digit and a symbol. The same rule is mirrored in the browser as a live checklist and strength bar.
- JWT sign-in (HS512, 24-hour expiry). Stateless logout: the browser simply discards the token.
- Password reset by an emailed link. The raw token is only ever emailed; the database keeps a SHA-256 hash. Links expire after 30 minutes.
- Sign-in rate limiting: 5 failed attempts per email and address, and 20 per address, in a 15-minute window; 10 registrations per address per hour.
- Identical error for an unknown email and a wrong password, and status shown only **after** a correct password, so the form cannot be used to discover who has an account.
- Profile page: change name and phone, change password, choose light, dark or match-my-device theme.

4.2 Account approval workflow

- Five account states: `PENDING`, `APPROVED`, `REJECTED` (with reason), `SUSPENDED` (with reason) and `DEACTIVATED`. Only approved accounts can sign in or use a token.
- Automatic routing: HOD → Principal; teacher → HOD of the department; student → class teacher of that department, semester and class, else the department HOD, else the Principal.
- Anyone higher in the chain may also act, so a request never waits on a missing approver.
- One approved HOD per department; a second HOD request is refused.
- Class teachers assigned per class by the HOD (or Principal/Admin); the page shows how many students each class has.
- A rejected applicant sees the reason after signing in and can correct details and apply again with the same email. A status card shows the full history of the account.
- Approvers see a badge with the number of requests waiting for them and receive an in-app notification.

4.3 Public website for visitors

- Home page with the two wings (Engineering and Polytechnic), latest public notices, admissions block and about section.
- Public notices page with search, category, priority and date filters; filters live in the URL so a filtered view can be shared.
- Notice page with attachments that open through a short-lived signed link, issued per click.
- Pages for Admissions, Engineering, Polytechnic, About and Help. College name, address and contact numbers come from configuration (`app.college.*`).
- Engineering and Polytechnic pages list their programmes and departments straight from the database.

4.4 Notices and targeting

- Seven target types: entire college, role, whole department, role in a department, semester, class, and specific users. Any number of audience rows can be combined on one notice.
- Which target types and roles are offered depends on the signed-in role; the server re-validates on save.
- Two visibility modes: **private** (to college members through targets) and **public** (also readable on the public website, for Engineering, Polytechnic or both). A public notice may also notify members.
- Three priorities (Normal, Important, Urgent), categories managed by the Super Admin, optional publish date, optional expiry date.
- Important and Urgent notices always require acknowledgement; Normal notices can opt in.
- Search, category, priority, department, semester, status, creator and date filters, plus server-side pagination, on every list.

4.5 Notice lifecycle and approval

- Eight notice statuses: `DRAFT`, `PENDING_APPROVAL`, `APPROVED`, `REJECTED`, `SCHEDULED`, `PUBLISHED`, `EXPIRED`, `ARCHIVED`.
- Every new notice, including AI-generated ones, starts as a draft.
- Principal, HOD and Admin publish directly (or schedule if the publish date is in the future).
- Faculty submit to the HOD; the HOD approves (publishing or scheduling in one step) or rejects with a reason; the faculty member edits and resubmits.
- Approval timeline on each notice, visible only to the creator, the reviewing HOD and the Super Admin.
- A notice that has passed its expiry cannot be published or approved; the author is told to set a new expiry.

4.6 Attachments

- PDF, DOC, DOCX, JPG, JPEG and PNG, up to 10 MB each and 10 per notice.
- Server-side checks: extension whitelist, MIME type match, size, and **file signature (magic bytes)**, so a renamed executable cannot pass as a PDF.
- Stored on Cloudinary with delivery type `authenticated`; the database keeps only metadata.
- Download through a signed link that expires after 5 minutes and is issued only if the caller may view the notice.
- Delete is "cloud first": if Cloudinary fails, the record is kept so no file is orphaned.
- If an upload batch fails halfway, files already pushed are removed again.

4.7 Notifications, email and read tracking

- In-app notification created for exactly the resolved recipients (active users, creator excluded) when a notice is published.
- Email only for Important and Urgent notices, one individual message per recipient, sent on a background thread pool so mail problems never break publishing.
- Notifications page grouped by day (Today, Yesterday, Earlier this week, Older) with an unread filter, mark-read, delete and "show older".
- A notice is marked as read the first time a recipient opens it; "I have read this notice" is a separate, explicit acknowledgement.

- Read and acknowledged counts are computed over **intended recipients only**, so a stray reader does not inflate the numbers.

4.8 Authoring experience

- **AI notice assistant**: type a plain sentence and receive a structured draft (title, formal body, category, priority, audience, dates). Uses the Anthropic API when a key is set, and an offline rule-based drafter otherwise.
- **Live recipient count** while choosing the audience, resolved by the same rules as saving, with a role breakdown.
- **Preview before publishing** and one-click "Save and publish" or "Save and submit for approval".
- **Unsaved work is kept in the browser** (per user and per notice) and offered for restore.
- **Drag-and-drop uploads** with per-file validation, per-file progress bar and per-file error.

4.9 Reader experience

- Role dashboards with counts and recent items; an **urgent banner** at the top of every dashboard.
- **Saved notices**: a personal reading list limited to notices the user can still see.
- Notice history (published and expired), unread filter, "needs acknowledgement" markers.
- Skeleton loading states, empty states that say what to do next, light and dark themes remembered per device, and a mobile bottom navigation bar.
- Skip-to-content link, ARIA live regions and labelled form fields for screen-reader and keyboard users.

4.10 Administration and quality

- User management (create, edit, activate, deactivate, suspend with reason), department management (with wing, programme and semester count) and category management (deactivate instead of delete when in use).
- Audit log with filters by action, user and entity type.
- Swagger UI with JWT support, a production profile that hides Swagger and reads secrets from environment variables, a Dockerfile for the backend, and a Compose file for MySQL.
- 72 automated tests, including integration tests for visibility, approval workflows, public-site leakage, registration routing and rate limiting.

§5 Technologies Used — what, why and where

For each technology: what it is, why it was chosen for this project specifically, and where in the codebase it appears. Versions are taken from `pom.xml` and `package.json`.

5.1 Backend technologies

Java 17

What it is: a statically typed, compiled, object-oriented language that runs on the JVM. Version 17 is a Long-Term Support release.

Why chosen: static typing catches whole classes of mistakes at compile time. Java 17 gives `record` types, which are used for all 55 request and response DTOs, and switch expressions, which keep the role-by-role rules in `TargetPolicy` short and readable.

Where used: the entire `backend/src/main/java/com/college/noticeboard` tree. Set by `<java.version>17</java.version>`.

Spring Boot 3.3.5

What it is: a framework that auto-configures the web server, database connection, JSON conversion and security from the dependencies you declare, and packages everything as one runnable JAR with an embedded Tomcat.

Why chosen: dependency injection let each of the 28 service classes be written and tested independently; starters turned email support into one dependency and a few properties; `@Scheduled` gave the publish and expire job a home with one annotation and no external scheduler.

Where used: everywhere. The entry point is `NoticeBoardApplication.java`, which carries `@EnableScheduling` and `@EnableAsync` and sets the default time zone to `Asia/Kolkata`.

Spring Security and JJWT 0.12.6

What it is: Spring Security is an authentication and authorisation framework built as a chain of servlet filters. JJWT creates and verifies JSON Web Tokens.

Why chosen: hand-written authentication is where student projects most often introduce real vulnerabilities. Spring Security provides tested BCrypt and declarative rules; `@EnableMethodSecurity` lets each endpoint carry its own `@PreAuthorize`. JWT keeps the API stateless, so no session store is needed.

Where used: `config/SecurityConfig.java` (filter chain, CORS, BCrypt bean), `config/JwtAuthenticationFilter.java` (validates the token on every request), `config/JwtService.java` (issues and parses tokens).

Spring Data JPA and Hibernate

What it is: JPA is the Java standard for object-relational mapping; Hibernate implements it; Spring Data JPA adds repositories that derive queries from method names and supports the `Specification` API for dynamic filters.

Why chosen: the notice list needs up to 12 optional filters combined with the visibility rule. `Specification` objects compose those at run time and run as one SQL query, with the pagination done by the database.

Where used: 12 entities, 12 repositories, and `service/NoticeSpecifications.java`. `ddl-auto=update` creates the tables on first start.

MySQL 8

What it is: a relational database. Connector/J 8 is the JDBC driver.

Why chosen: the data is strongly relational (users, departments, notices, targets, reads) and the project depends on unique constraints, foreign keys and indexes. A Compose file starts MySQL 8.4 in one command.

Where used: `application.properties` (`spring.datasource.*`), `docker-compose.yml`. Time zone is pinned to `Asia/Kolkata` in both the JDBC URL and Hibernate.

Bean Validation (Jakarta)

What it is: annotations such as `@NotBlank`, `@Size`, `@Pattern`, `@Future` that declare rules on request objects.

Why chosen: validation rules live next to the fields, run before the controller body, and produce a field-by-field error map automatically.

Where used: all DTO records, for example `AuthDtos.PASSWORD_REGEX` and `NoticeDtos.NoticeRequest`.

Cloudinary 1.36

What it is: a cloud media service with an upload API and signed delivery URLs.

Why chosen: uploaded files must not live on the application server's disk, and they must not be public. Cloudinary's `authenticated` delivery type produces URLs that do not work without a signature.

Where used: `service/CloudinaryService.java` behind the `CloudStorageService` interface, which lets tests replace it with a mock.

Spring Mail (Gmail SMTP)

What it is: Spring's `JavaMailSender` over SMTP.

Why chosen: Important and Urgent notices must reach people outside the browser. It is optional: when `MAIL_ENABLED=false` the application logs and carries on.

Where used: `service/EmailService.java`, with a dedicated thread pool `mailExecutor` in `config/AsyncConfig.java`.

Anthropic Messages API (via Spring `RestClient`)

What it is: an HTTP API for a large language model.

Why chosen: typing a notice from scratch is slow. The assistant turns one plain sentence into a structured draft. A rule-based fallback keeps the feature working with no key.

Where used: `AnthropicAiSuggestionClient`, `MockAiSuggestionClient`, `AiNoticeService`, `AiController`. Settings: `app.ai.*`.

springdoc-openapi 2.6 (Swagger UI)

What it is: generates OpenAPI documentation from the controllers.

Why chosen: interactive, always-current API documentation with an Authorize button for the JWT. Switched off in the production profile.

Where used: `config/OpenApiConfig.java`; UI at `/swagger-ui.html`.

Lombok 1.18.42

What it is: a compile-time annotation processor that writes getters, setters, constructors and builders.

Why chosen: less boilerplate in entities. The version is pinned in `pom.xml` because the one bundled with Spring Boot 3.3 crashes `javac` on JDK 23 and newer.

Where used: entity classes and services (`@RequiredArgsConstructor`, `@Builder`, `@Getter`).

JUnit 5, Mockito, Spring Security Test and H2

What it is: the test stack.

Why chosen: integration tests run the real controllers, security and JPA queries against an in-memory H2 database in MySQL mode, with Cloudinary mocked, so the suite needs no external service.

Where used: `backend/src/test` and `application-test.properties`.

5.2 Frontend technologies

React 18.3 and Vite 5.4

What it is: React builds user interfaces from components. Vite is a development server and bundler with instant hot-reload.

Why chosen: components such as `NoticeCard`, `TargetBuilder` and `FileDrop` are reused across many pages, and hooks (`useAsync`, `useAuth`) keep data-loading logic out of the pages. Vite starts in a second and builds with one command.

Where used: the whole `frontend/src` tree; `vite.config.js` sets the dev port to 5173.

React Router 6.27

What it is: client-side routing.

Why chosen: one URL space for two experiences: `/` shows the public home page to visitors and the dashboard to signed-in members. Nested routes let one layout wrap many pages.

Where used: `App.jsx` (29 routes), `RootFrame`, `ProtectedRoute`.

Axios 1.7

What it is: an HTTP client with interceptors.

Why chosen: one request interceptor attaches `Authorization: Bearer <token>` to every call, and one response interceptor signs the user out on a 401. Upload progress for the file drop comes from `onUploadProgress`.

Where used: `services/api.js`.

Bootstrap 5.3 and a custom stylesheet

What it is: a CSS framework, plus about 450 lines of project CSS.

Why chosen: Bootstrap provides forms, modals and the grid; the custom CSS gives the notice-board look, the priority pins, the public site, and a full dark theme driven by `data-bs-theme`.

Where used: `main.jsx` imports Bootstrap; `styles.css` holds the theme tokens.

Browser `localStorage`

What it is: per-browser key-value storage.

Why chosen: three small, non-sensitive uses: the JWT (`cnb_token`), the theme (`cnb_theme`), and unsaved notice drafts (`cnb_draft:<user>:<notice>`).

Where used: `services/api.js`, `utils/theme.js`, `pages/NoticeForm.jsx`.

5.3 External services and tools

Tool	Role	Required?
Cloudinary	Attachment storage and signed download links	Optional. Without it the app runs and uploads return a clear error.
Gmail SMTP	Emails for Important/Urgent notices, password reset, account decisions	Optional (`MAIL_ENABLED`).
Anthropic API	AI notice drafting	Optional (`AI_API_KEY`). Offline fallback exists.
Docker / Docker Compose	Backend image and a local MySQL 8.4	Optional.
Maven wrapper (`mvnw`)	Builds the backend without installing Maven	Bundled.
Git	Version control	—

§6 Complete Architecture

The system uses a classic **three-tier architecture**, and within the backend a strict **layered architecture**. A layer may only call the layer directly beneath it.

6.1 The three tiers

Tier	Technology	Responsibility
Presentation	React 18 + Vite in the browser	Render the interface, collect input, validate it for fast feedback, hold the token and call the API. Contains no authoritative rule.
Application	Spring Boot 3.3 on port 8080	Authenticate and authorise, enforce every business rule, orchestrate Cloudinary, mail and AI, and translate between DTOs and entities. The only trusted tier.
Data	MySQL 8 + Cloudinary	Persist the data and enforce structural integrity through primary keys, foreign keys and unique indexes. Cloudinary holds the file bytes.

> **Why the client's validation and hiding do not count**
> Every rule in the React app exists again in the backend. The client version exists only so the user gets instant feedback and a clean interface. Anyone can bypass the browser and call the API with `curl`, so the server re-validates and re-authorises everything. The duplication is deliberate.

6.2 The backend's layers

```
HTTP request
    |
    v
1. SECURITY FILTER CHAIN     config/SecurityConfig, JwtAuthenticationFilter
   CORS -> JWT filter -> URL rules. Rejects unauthenticated requests here.
    |  (SecurityContext now holds a UserPrincipal)
    v
2. CONTROLLER LAYER          controller/*.java   (13)
   Maps URL + method to a Java method. Runs @Valid on the body and
   @PreAuthorize on the role. Contains NO business logic.
    |  (DTO in, DTO out)
    v
3. SERVICE LAYER             service/*.java      (28 classes and interfaces)
   ALL business rules: targeting, approval routing, lifecycle, access checks,
   scheduling, notifications. Transaction boundaries (@Transactional).
    |  (entities)
    v
4. REPOSITORY LAYER          repository/*.java   (12)
   Spring Data interfaces + Specifications. The only layer that talks to MySQL.
    |
    v
5. DATABASE                  MySQL 8, 12 tables
```

Two cross-cutting pieces sit beside the layers. `GlobalExceptionHandler` turns every exception into one JSON error shape. `AuditService` is called from services to record who did what.

6.3 The authorisation stack: four checkpoints, not one

A notice request passes through four independent checks. Removing any one of them would still leave the others standing, which is the point.

#	Checkpoint	Where	Example
1	Is there a valid token for an **approved** account?	`JwtAuthenticationFilter`	A suspended user's old token stops working immediately because the filter reloads the account.
2	Is the **role** allowed to call this endpoint?	`@PreAuthorize` on the controller	A student calling `POST /api/notices` gets 403.
3	Is this **audience** allowed for this role?	`TargetPolicy`	An HOD targeting another department gets 403 "HODs can only target their own department".
4	May this **user see this notice**?	`NoticeAccessService`	A student opening `/api/notices/42` for a different class gets 403.

6.4 How a notice reaches a reader

```
Author (React NoticeForm)
   | POST /api/notices  (JSON: title, text, category, priority, dates, targets[])
   v
NoticeController --@Valid--> NoticeService.create()
   |-- TargetPolicy.buildTargets()     validate + authorise every audience row
   |-- build Notice (status = DRAFT) and its NoticeTarget rows
   |-- AuditService.log(NOTICE_CREATED)
   v
Author clicks "Save and publish"  ->  POST /api/notices/{id}/publish
   |-- NoticeAccessService.canPublish()
   |-- publish date in the future?  yes -> SCHEDULED  (the scheduler publishes later)
   |                                no  -> publishNow()
   |-- NotificationService.notifyPublished()
   |      RecipientService resolves active users matching ANY target row
   |      -> one Notification row per recipient
   |      -> if IMPORTANT/URGENT: EmailService.sendToEach()  (async thread pool)
   v
Reader (React NoticeList)
   | GET /api/notices?scope=feed
   v
NoticeService.list(): published AND live AND NoticeSpecifications.targetedTo(me)
   -> SQL EXISTS(sub-query on notice_targets that matches my role/dept/sem/class/id)
```

6.5 Public website: a separate, read-only door

Visitors never touch the authenticated endpoints. `/api/public/**` is the only anonymous surface and it is `GET` only. `PublicNoticeService` starts **every** query from `NoticeSpecifications.publiclyVisible(now)` and re-checks `Notice.isPubliclyVisible(now)` on every single-item lookup. Responses use dedicated DTOs (`PublicDtos`) that contain no creator id, no targets, no status and no review history.

§7 Complete Folder Structure

7.1 Repository layout

```
college-notice-board/
|-- README.md                  Project readme (features, roles, API, setup)
|-- docker-compose.yml         Local MySQL 8.4 (development DB user)
|-- .env.example               Every environment variable, documented
|-- backend/                   Spring Boot application
|   |-- pom.xml                Dependencies and build
|   |-- Dockerfile             Two-stage build (Maven -> JRE 17), prod profile
|   |-- mvnw, mvnw.cmd         Maven wrapper
|   `-- src/
|       |-- main/java/com/college/noticeboard/
|       |   |-- NoticeBoardApplication.java
|       |   |-- config/        13  security, JWT, CORS, Swagger, async, seeder, upgrades
|       |   |-- controller/    13  REST endpoints (no business logic)
|       |   |-- service/       28  business rules, visibility, storage, mail, AI, scheduler
|       |   |-- repository/    12  Spring Data repositories
|       |   |-- entity/        22  12 JPA entities, 9 enums, 1 base class
|       |   |-- dto/           18  55 request/response records + mappers
|       |   `-- exception/     11  typed exceptions + global handler
|       |-- main/resources/
|       |   |-- application.properties        development settings
|       |   `-- application-prod.properties   production (env variables)
|       `-- test/java/com/college/noticeboard/   8 test classes, 72 tests
`-- frontend/                  React single-page application
    |-- index.html             Applies saved theme before first paint
    |-- package.json, vite.config.js, .env.example
    `-- src/
        |-- main.jsx           Providers: Router > Theme > Toast > Auth
        |-- App.jsx            All 29 routes
        |-- styles.css         Theme tokens, layout, public site, dark mode
        |-- services/api.js    Axios instance + 12 API modules
        |-- context/           AuthContext, ThemeContext, ToastContext
        |-- hooks/             useAsync, usePublicData
        |-- utils/             format.js, theme.js
        |-- components/        18 reusable components
        `-- pages/             19 page files (+ public/ and admin/ folders)
```

7.2 Backend packages in one line each

Package	Holds	Rule it follows
config	`SecurityConfig`, `JwtService`, `JwtAuthenticationFilter`, `UserPrincipal`, `CurrentUserService`, `DataSeeder`, `InstitutionUpgrade`, `AccountStatusBackfill`, `CollegeInfo`, `OpenApiConfig`, `AsyncConfig`	Wiring and start-up work. No business decisions.
controller	Auth, Users, Departments, Categories, Notices, Attachments, Notifications, Dashboard, Registrations, ClassTeachers, AuditLogs, Public, Ai	Thin: map the URL, validate the body, delegate.
service	`NoticeService`, `TargetPolicy`, `TargetMatcher`, `NoticeAccessService`, `RegistrationService`, `AuthService`, `AuthRateLimiter`, `AttachmentService`, `FileValidator`, `CloudinaryService`, `NotificationService`, `EmailService`, `AiNoticeService`, `PublicNoticeService`, …	Every rule lives here, inside `@Transactional` boundaries.
repository	One interface per table, plus custom `@Query` methods	Never contains rules.
entity	JPA entities and enums	Mirrors the tables; small helper methods only.
dto	Request and response `records`, `NoticeMapper`, `UserMapper`, `SimpleMappers`	Entities are never returned from the API.
exception	`ApiException` and subclasses, `GlobalExceptionHandler`	One HTTP status per exception type.

7.3 Frontend structure in one line each

Folder	Holds
pages/	`Dashboard`, `NoticeList`, `NoticeDetails`, `NoticeForm`, `Notifications`, `AccountRequests`, `ClassTeachers`, `Profile`, `AuthPages` (Login, Register, ForgotPassword, ResetPassword)
pages/public/	`Landing`, `PublicNotices`, `PublicNoticeDetail`, `InstitutionPage`, `InfoPages` (Admissions, About, Help)
pages/admin/	`Users`, `Departments`, `Categories`, `AuditLogs`, `SimpleCrud` (shared table-and-form)
components/	`Layout`, `PublicLayout`, `ProtectedRoute`, `NoticeCard`, `PublicNoticeCard`, `TargetBuilder`, `AudienceSummary`, `FileDrop`, `PasswordField`, `UrgentBanner`, `NoticeTimeline`, `AccountStatusCard`, `SaveButton`, `ConfirmDialog`, `Pagination`, `States`, `ThemeControl`, `InstitutionIcon`
context/	Signed-in user, theme preference, toast messages
hooks/	`useAsync` (load, error, reload) and shared cached public data
utils/	Labels, date helpers and file-size formatting; theme helpers

§8 Frontend — pages, components, routing, state, validation

The frontend is a single-page React application. It has two faces on one URL space: a **public site** for visitors, and a **portal** for signed-in members. Nothing in it is trusted; it mirrors rules the backend enforces.

8.1 Start-up and providers

`main.jsx` wraps the app in four providers, outermost first: `BrowserRouter` → `ThemeProvider` → `ToastProvider` → `AuthProvider` → `App`. `index.html` contains a small inline script that reads the saved theme from `localStorage` and sets `data-bs-theme` **before** the first paint, so dark mode never flashes white.

Context	State it holds	Used for
`AuthContext`	`user`, `loading`, `login()`, `logout()`, `setUser()`. On load, if a token exists it calls `GET /users/me` to restore the session.	Every page that needs to know who is signed in, and every role-based show/hide.
`ThemeContext`	`pref` (system, light, dark), `resolved` (light or dark), `choose()`. Watches the operating-system preference.	The theme toggle and the Profile page.
`ToastContext`	A list of messages that disappear after 4.5 seconds, drawn in a live region.	Success and error feedback after every action.

8.2 Routing: one URL space, two faces

`App.jsx` declares 29 routes. The root route renders `RootFrame`, which picks the frame: `Layout` (sidebar portal) when a user is signed in, `PublicLayout` (header, footer with contact details) when not. The index route renders `Home`, which is the **dashboard** for members and the **landing page** for visitors.

Group	Routes	Access
Auth (own frame)	`/login` `/register` `/forgot-password` `/reset-password`	Anyone
Public site	`/` (visitors) `/public-notices` `/public-notices/:id` `/admissions` `/engineering` `/polytechnic` `/about` `/help`	Anyone; also reachable inside the portal
Member portal	`/` (dashboard) `/notices` `/saved` `/history` `/notices/:id` `/notifications` `/profile`	Any signed-in role
Creators	`/my-notices` `/notices/new` `/notices/:id/edit` `/account-requests`	Super Admin, Principal, HOD, Faculty
HOD only	`/approvals`	HOD
Class management	`/class-teachers`	Super Admin, Principal, HOD
Administration	`/admin/all-notices` `/admin/users` `/admin/departments` `/admin/categories` `/admin/audit-logs`	Super Admin
Fallback	*	Redirects to `/`

`ProtectedRoute` redirects an anonymous visitor to `/login` (remembering where they were going) and redirects a signed-in user without the right role to `/`. It is a **convenience**: the backend rejects the same request independently.

8.3 The API layer (`services/api.js`)

- One Axios instance with base URL `VITE_API_URL` (default `http://localhost:8080/api`).
- **Request interceptor**: attaches `Authorization: Bearer <token>` when a token is stored.
- **Response interceptor**: on a 401 *while holding a token*, clears the token and redirects to `/login?expired=1`. A failed sign-in (no token yet) is left alone so its error can be shown.
- `errorMessage(err)` turns any failure into one human sentence: joined field errors, then the server's `message`, then "Cannot reach the server..." when there was no response.
- `fieldErrors(err)` returns the `{ field: message }` map so forms can mark individual inputs.
- Twelve modules (`AuthApi`, `UserApi`, `DepartmentApi`, `CategoryApi`, `NoticeApi`, `AttachmentApi`, `NotificationApi`, `DashboardApi`, `PublicApi`, `RegistrationApi`, `ClassTeacherApi`, `AuditApi`), each returning `response.data` so pages never touch Axios directly.
- `AttachmentApi.uploadOne()` sends **one file per request** so each file has its own progress and error.

8.4 Pages

Page	What it does and the interesting detail
`AuthPages` — **Login**	Client checks (email shape, password present). On 403 with an `account` object it swaps to `AccountStatusCard` instead of showing an error. 'Check again' re-submits.
`AuthPages` — **Register**	Role chooser (Student, Teacher, HOD) with a hint on who approves. Institution is chosen first; only that wing's departments and semester count are offered. Students add semester and class. Password checklist and confirm field. Server field errors are mapped back onto the inputs.
`Dashboard`	Loads `/dashboard/<role>`. Shows `UrgentBanner`, a row of count tiles (tiles needing attention are highlighted), recent notices, read statistics and a department breakdown.
`NoticeList`	One component for six scopes: `feed`, `history`, `mine`, `approvals`, `saved`, `all`. Filter panel (search, category, priority, department, semester, status, unread only), server pagination, skeleton loading, an empty state per scope.
`NoticeForm`	The largest page (≈ 21 KB). Create and edit; AI draft box; title, text, category, priority, dates, acknowledgement; private/public visibility; `TargetBuilder`; `FileDrop`; browser auto-save with restore; a **preview step** with live `AudienceSummary`; buttons that adapt to role (Publish for Principal/HOD/Admin, Submit for approval for Faculty).
`NoticeDetails`	Renders the notice and, from the server's `permissions` object, only the buttons the caller may use: Edit, Publish, Submit, Approve, Reject, Archive, Delete, Save, Acknowledge, statistics. Shows rejection reason, attachments (signed download), and `NoticeTimeline`.
`Notifications`	Grouped by day, unread filter, open (marks read and navigates), mark read, delete, mark all read, 'show older'.
`AccountRequests`	Approver's queue split into 'waiting for you' and 'others'. Approve, or Reject with a reason through `ConfirmDialog`.
`ClassTeachers`	One row per class (from student counts plus assignments); pick a teacher from a dropdown; add a class manually; remove an assignment.
`Profile`	Edit name and phone, change password, choose theme.
`admin/Users`	Search, role/department/status filters, create/edit modal, close, reopen, approve, suspend with reason.
`admin/Departments`, `Categories`	`SimpleCrud`: a table plus an inline form. Departments carry wing, programme and semester count.
`admin/AuditLogs`	Filter by action, user and entity type, paginated.
`public/*`	Landing, PublicNotices (filters in the URL), PublicNoticeDetail (signed link per attachment click), InstitutionPage (programmes from the DB), Admissions, About, Help.

8.5 Components

Component	Purpose
`Layout`	Portal frame: sidebar with role-aware links, top bar with theme toggle and notification button, mobile **bottom navigation**. Polls the unread count and pending-request count every 60 seconds.
`PublicLayout`	Visitor frame: brand, navigation, Login button, footer with address, phone, email. Sets the page title to the college name.
`TargetBuilder`	Builds the audience list. Offers only the target types and roles the server said are allowed for this role (`/notices/target-options`) and pins the department for HOD and Faculty.
`AudienceSummary`	Debounced (400 ms) call to `/notices/recipient-preview`; shows 'N people will receive this notice' with a role breakdown and warns when the audience is empty.
`FileDrop`	Drag-and-drop zone and per-file list. Rejects wrong type, size over 10 MB and empty files before upload, mirroring `FileValidator`.
`PasswordField`	Show/hide toggle, a rule checklist whose regexes mirror `PASSWORD_REGEX`, and a 0–4 strength bar.
`UrgentBanner`	Fetches up to 3 active urgent notices for me; read ones are dimmed; disappears by itself.
`NoticeTimeline`	Lifecycle history from `/notices/{id}/timeline` plus a 'next step' line.
`AccountStatusCard`	What a not-yet-approved person sees: status, who it waits for, reason, history, and a 'correct details and apply again' link for rejected applicants.
`SaveButton`	Optimistic toggle for the reading list; rolls back if the server refuses.
`ConfirmDialog`	Modal confirmation; with `inputLabel` it requires typed text (the rejection reason).
`NoticeCard`, `PublicNoticeCard`	List items. Priority is shown by a coloured pin strip and a label, never by colour alone.
`States`	`Loading`, `Empty`, `ErrorState` and four skeleton placeholders shaped like the real content.
`Pagination`, `ThemeControl`, `InstitutionIcon`	Small shared pieces.

8.6 State management

There is no Redux. State is kept at the smallest scope that works.

- **Global**: the signed-in user, theme and toasts (three contexts).
- **Per page**: filters, page number and form values in `useState`.
- **Server data**: `useAsync(loader, deps)` returns `{ data, loading, error, reload }` and re-runs when its dependencies change.
- **Shared and rarely changing**: `usePublicData` caches the college details, wings and categories for the life of the page load.
- **Persisted**: token, theme preference, and unsaved notice drafts in `localStorage`.

8.7 Validation on the client

Form	Client checks	Always re-checked by
Login	Email shape, password present	`LoginRequest`, rate limiter
Register	Name, email shape, optional phone `7–20` chars, institution, department, semester and class (students), password rules, confirm match	`RegisterRequest` + `AuthService.register`
Notice	Title ≤ 200, text ≤ 20,000, required category, expiry in future (browser date inputs)	`NoticeRequest` (`@Valid`, `@Future`), `NoticeService.validateDates`, `TargetPolicy`
Attachments	Extension, 10 MB, non-empty	`FileValidator` (also MIME and signature)
Password change / reset	Same checklist as registration	`PASSWORD_REGEX`

8.8 Accessibility and responsiveness

- Skip-to-content link on the public site; `aria-live` regions for toasts, status cards and the audience summary; `aria-invalid`/`aria-describedby` on invalid fields; visible focus outline.
- Priority and status are always text as well as colour.
- Under 860 px the sidebar becomes a slide-in drawer and a bottom navigation bar appears.
- Light and dark themes via Bootstrap's `data-bs-theme` and CSS variables.

§9 Backend — packages, classes and the request journey

9.1 The journey of one request: `POST /api/notices/42/publish`

1. The request arrives with `Authorization: Bearer <jwt>`. Spring's filter chain runs `JwtAuthenticationFilter`.
2. The filter extracts the token, `JwtService.validateAndGetSubject()` verifies the HS512 signature and expiry and returns the email.
3. `CustomUserDetailsService` loads the user from MySQL **on every request**. If the account is not approved (`isEnabled()` false) no authentication is set, so a suspended user's old token dies at once.
4. `SecurityConfig` rules: `/api/auth/**` and `GET /api/public/**` are open; everything else needs authentication. Method security then applies `@PreAuthorize("hasAnyRole('SUPER_ADMIN','PRINCIPAL','HOD')")`.
5. `NoticeController.publish(42)` calls `NoticeService.publish`.
6. `CurrentUserService.currentUser()` reloads the user from the database, so role and department are authoritative and never taken from the token.
7. `NoticeAccessService.canPublish()` requires: caller is the creator, role is Admin, Principal or HOD, and the notice is a `DRAFT`.
8. `releaseOrSchedule`: if the expiry has passed, fail; if the publish date is in the future, set `SCHEDULED`; else `publishNow`.
9. `publishNow` sets `PUBLISHED`, records `publishedAt`, calls `NotificationService.notifyPublished` and writes a `NOTICE_PUBLISHED` audit entry with the recipient count.
10. The transaction commits and a `NoticeDetailResponse` DTO (with a `permissions` object) is returned as JSON.

9.2 Configuration classes (`config/`)

Class	Job
`SecurityConfig`	Stateless chain, CSRF off (see §15), CORS from `app.cors.allowed-origins`, public routes, `BCryptPasswordEncoder`, JSON 401/403 handlers, JWT filter registered once.
`JwtService`	Builds the HS512 key; **fails at start-up if the secret is under 64 bytes**. Issues a token with subject = email and claims `uid` and `role`; 24 h expiry.
`JwtAuthenticationFilter`	`OncePerRequestFilter` described above.
`UserPrincipal`, `CustomUserDetailsService`	Adapt the `User` entity to Spring Security; role becomes `ROLE_<NAME>`.
`CurrentUserService`	`currentUserId()` from the token; `currentUser()` loaded fresh from the DB.
`JsonSecurityHandlers`	Writes 401 'Authentication required' and 403 'You do not have permission…' in the API's JSON shape.
`AsyncConfig`	`mailExecutor`: 2 core / 4 max threads, queue of 5,000.
`CollegeInfo`	Binds `app.college.*` for the public site.
`OpenApiConfig`	Swagger title, Bearer scheme.
`DataSeeder`	Runs only if `app.seed.enabled=true` **and** the users table is empty. Creates 9 departments, 17 categories, 14 active users and 2 pending requests, 2 class teachers, 6 public notices, and private notices (9 published, 1 pending approval, 1 scheduled).
`InstitutionUpgrade`, `AccountStatusBackfill`	Idempotent start-up upgrades for older databases (see §14.9).

9.3 Controllers (`controller/`)

Controllers are deliberately thin: each method is one or two lines that delegate to a service. Annotated with springdoc `@Tag`/`@Operation`. The complete list of endpoints is in §11.

Controller	Base path	Endpoints
`AuthController`	`/api/auth`	5
`UserController`	`/api/users`	12
`DepartmentController`	`/api/departments`	4
`CategoryController`	`/api/categories`	4
`NoticeController`	`/api/notices`	18
`AttachmentController`	`/api` (notices and attachments)	4
`NotificationController`	`/api/notifications`	5
`DashboardController`	`/api/dashboard`	5
`RegistrationController`	`/api/registrations`	4
`ClassTeacherController`	`/api/class-teachers`	4
`AuditLogController`	`/api/audit-logs`	1
`PublicController`	`/api/public`	6
`AiController`	`/api/ai`	1

9.4 Services (`service/`) — the core

`NoticeService` — the lifecycle engine (≈ 25 KB)

- `create` / `update`: validates dates, picks an **active** category, applies visibility and targets, always starts as `DRAFT`. Editing a `REJECTED` notice turns it back into a draft.
- `applyVisibilityAndTargets`: private notices need at least one target; public notices need a wing, and HODs and teachers may only choose their own department's wing.
- `publish`, `submit`, `approve`, `reject`, `archive`, `delete`: each checks `NoticeAccessService` first, then changes state, writes an audit entry and notifies the right people.
- `processDueNotices(now)`: called by the scheduler. Publishes due `SCHEDULED` notices (or expires them if their expiry already passed) and expires `PUBLISHED` notices past their expiry.
- `list`: builds a JPA `Specification` per scope, adds the optional filters, and pages in the database.
- `summaries`: loads attachment counts, read flags and saved flags for a whole page in three batched queries (no N+1).
- `acknowledge`, `save/unsave`, `markRead`, `stats/statsFor`, `previewRecipients`, `targetOptions`.

Visibility and access

- `TargetPolicy`: `normalise` (fills required fields per type, upper-cases class names), `authorise` (per-role rules), `optionsFor` (what the UI may offer), `defaultTargetFor` (fallback for AI drafts). Duplicate rows are collapsed.
- `TargetMatcher`: static `matches` and `matchesAny` — the Java form of the visibility rule.
- `NoticeSpecifications`: the SQL form (`targetedTo`, `usersMatching`) plus every filter, `activeAt`, `publiclyVisible`, `savedBy`, `notReadBy`, and so on.
- `NoticeAccessService`: `canView`, `canEdit`, `canDelete`, `canPublish`, `canSubmit`, `canReview`, `canAcknowledge`, `canViewStats`, `canArchive`, `canSave`, and `permissions()`, which packs them into the object the UI uses.
- `RecipientService`: resolves the exact set of **active** users matching any target row; **no targets means nobody**, never everybody.

Accounts and approval

- `AuthService`: register, login, forgot and reset password (see §12).
- `RegistrationService`: approver routing, `canReview`, queue, approve, reject, HOD vacancy check, and the applicant's status view (§14.2).
- `AuthRateLimiter`: in-memory sliding windows keyed by email+IP and by IP.
- `UserService`, `ClassTeacherService`, `DepartmentService`, `CategoryService`.

Files, messages, AI, reporting

- `AttachmentService`, `FileValidator`, `CloudStorageService` (interface) and `CloudinaryService`.
- `NotificationService`, `EmailService`.
- `AiNoticeService` with `AnthropicAiSuggestionClient` and `MockAiSuggestionClient`.
- `PublicNoticeService` (everything visitors see), `DashboardService`, `NoticeTimelineService`, `AuditService`, `NoticeScheduler`.

9.5 Entities (`entity/`)

Entity	Notes
`User`	Extends `BaseTimestamps`. Holds `status` and keeps the older `active` flag in step with it (`active == (status == APPROVED)`) through `setStatus`, `setActive` and a `@PrePersist/@PreUpdate` hook.
`Department`	Name unique **per wing**, code unique overall. Getters fall back to sensible defaults (Engineering → B.Tech, 8 semesters; Polytechnic → Diploma, 6) for old rows.
`Notice`	Holds `visibility`, `institutionScope`, flags, review fields, and a cascading list of targets. Helper `isPubliclyVisible(now)` is the single Java definition of 'public and live'.
`NoticeTarget`	One audience row with nullable role, department, semester, class and user.
`ClassTeacher`	One teacher per (department, semester, class).
`Notification`	Message, optional notice, optional in-app link, read flag.
`NoticeRead`, `NoticeAcknowledgement`, `SavedNotice`	One row per (notice, user), enforced by unique constraints.
`Attachment`	Cloudinary public id, URL, resource type, format, size, uploader.
`AuditLog`	Actor id and email (null id means the system), action, entity type and id, description.
`NoticeCategory`	Name and an `active` flag.

Enums: `Role`, `AccountStatus`, `NoticeStatus`, `Priority`, `TargetType`, `NoticeVisibility`, `InstitutionType`, `InstitutionScope` and `AuditAction` (27 values). They are stored as strings (`@Enumerated(EnumType.STRING)`) so that re-ordering the enum can never corrupt data.

9.6 DTOs and mappers (`dto/`)

55 `record`s in 18 files. The rule: **entities never leave the service layer.** `NoticeMapper` builds summaries and details, creates the human-readable audience labels ('Computer Engineering semester 6 class A'), and builds the 180-character excerpt. `PublicDtos` has separate records for visitors with no internal fields. `PageResponse<T>` is the common wrapper: `content, pageNumber, pageSize, totalElements, totalPages, last`.

9.7 Exceptions (`exception/`)

Exception	HTTP	Typical cause
`BadRequestException`, `NoticeApprovalException`	400	Invalid state or input, e.g. expiry has passed
`UnauthorizedException`	401	No valid authentication
`ForbiddenException`, `AccountStatusException`	403	Not allowed; account not approved
`ResourceNotFoundException`	404	Unknown id (also used for non-public notices on the public API)
`TooManyRequestsException`	429	Rate limit
`CloudinaryUploadException`	502	Cloud storage failed

9.8 Scheduler and asynchronous work

`NoticeScheduler.run()` is annotated `@Scheduled(fixedDelayString = "${app.scheduler.interval-ms:60000}", initialDelay = 10000)`: it starts 10 seconds after boot and then runs one minute after the previous run finishes. It is wrapped in try/catch so one failure cannot stop future runs. `EmailService.sendToEach` is `@Async("mailExecutor")`, so publishing returns to the user before any email is sent.

§10 Database — every table, every column, every relationship

Schema `college_notice_board`, twelve tables, created automatically by Hibernate from the entity classes on first start (`spring.jpa.hibernate.ddl-auto=update`). Column names are the snake_case form of the Java field names. `data.sql` is never executed (`spring.sql.init.mode=never`).

10.1 users

Indexes: `idx_user_role (role)` and `idx_user_dept_sem_class (department_id, semester, class_name)`, which back the recipient queries.

Column	Type	Meaning
`id`	`BIGINT PK AI`	Primary key
`full_name`	`VARCHAR(120) NOT NULL`	Display name
`email`	`VARCHAR(160) UNIQUE NN`	Login identifier. Stored lower-case; compared case-insensitively
`password`	`VARCHAR(255) NN`	**BCrypt hash**, never the plain password
`role`	`VARCHAR(20) NN`	SUPER_ADMIN, PRINCIPAL, HOD, FACULTY or STUDENT (stored as text)
`department_id`	`BIGINT FK NULL`	Department. Required for HOD, Faculty and Student; optional for Admin and Principal
`semester`	`INT NULL`	Students only
`class_name`	`VARCHAR(20) NULL`	Students only. Stored upper-case ("A")
`phone`	`VARCHAR(20) NULL`	Optional contact number
`active`	`BOOLEAN NN`	Kept in step with `status`: true exactly when status = APPROVED
`status`	`VARCHAR(20)`	PENDING, APPROVED, REJECTED, SUSPENDED, DEACTIVATED. Null in old rows is read from `active`
`status_reason`	`VARCHAR(500)`	Why the account was rejected or suspended; shown at sign-in
`status_reviewed_by`	`BIGINT FK NULL`	The person who approved, rejected or suspended
`status_reviewed_at`	`DATETIME`	When
`reset_token_hash`	`VARCHAR(128)`	**SHA-256 of** the password-reset token. The raw token exists only in the email
`reset_token_expiry`	`DATETIME`	30 minutes after the request
`created_at, updated_at`	`DATETIME NN`	Set by `@PrePersist` / `@PreUpdate` in `BaseTimestamps`

10.2 departments

Constraint: `uk_department_name_institution (name, institution_type)`.

Column	Type	Meaning
`id`	`BIGINT PK AI`	Primary key

Column	Type	Meaning
name	VARCHAR(120) NN	e.g. "Computer Engineering". The same name may exist in both wings
code	VARCHAR(20) UNIQUE NN	e.g. CS, DCO. Unique across the whole college
description	VARCHAR(500)	Shown on the public wing pages
institution_type	VARCHAR(20)	ENGINEERING or POLYTECHNIC. Null in old rows is treated as ENGINEERING
program_name	VARCHAR(60)	"B.Tech" or "Diploma" (defaults by wing)
semester_count	INT	8 for B.Tech, 6 for Diploma (defaults by wing); drives the registration form

10.3 class_teachers

Constraint: `uk_class_teacher_class (department_id, semester, class_name)` - one teacher per class.

Column	Type	Meaning
id	BIGINT PK AI	Primary key
department_id	BIGINT FK NN	Department of the class
semester	INT NN	Semester of the class
class_name	VARCHAR(20) NN	Division, upper-case
teacher_id	BIGINT FK NN	The approved teacher responsible for the class
assigned_at	DATETIME NN	When assigned

10.4 notice_categories

Column	Type	Meaning
id	BIGINT PK AI	Primary key
name	VARCHAR(80) UNIQUE NN	Academic, Examination, Placement, Holiday, ...
description	VARCHAR(300)	Optional
active	BOOLEAN NN	Inactive categories cannot be chosen for new notices; a category in use is deactivated, not deleted

10.5 notices

Index: `idx_notice_status (status)`, used by the scheduler and by every list.

Column	Type	Meaning
id	BIGINT PK AI	Primary key
title	VARCHAR(200) NN	Headline
description	TEXT NN	Body (up to 20,000 characters by validation)
category_id	BIGINT FK NN	Category

Column	Type	Meaning
priority	VARCHAR(20) NN	NORMAL, IMPORTANT, URGENT
created_by	BIGINT FK NN	Author
department_id	BIGINT FK NULL	Author's department **at creation time**; used to route faculty notices to the right HOD
publish_date	DATETIME	When it should go live; null means as soon as published
expiry_date	DATETIME	When it stops being active; null means never
published_at	DATETIME	When it actually went live
status	VARCHAR(20) NN	DRAFT, PENDING_APPROVAL, APPROVED, REJECTED, SCHEDULED, PUBLISHED, EXPIRED, ARCHIVED
rejection_reason	VARCHAR(1000)	Set by the HOD on rejection
reviewed_by, reviewed_at	BIGINT FK / DATETIME	The reviewing HOD and when
acknowledgement_required	BOOLEAN NN	Lets a NORMAL notice ask for acknowledgement. IMPORTANT and URGENT always do
ai_generated	BOOLEAN NN	True when created by the AI assistant (shown as 'Drafted with AI')
visibility	VARCHAR(10)	PRIVATE (default) or PUBLIC. Null in old rows means PRIVATE
institution_scope	VARCHAR(20)	For public notices: ENGINEERING, POLYTECHNIC or BOTH
created_at, updated_at	DATETIME NN	Timestamps

10.6 notice_targets

Index: `idx_target_notice (notice_id)`. **A row matches a user when every non-null column equals the user's value.** A row of all nulls (type COLLEGE) therefore matches everyone.

Column	Type	Meaning
id	BIGINT PK AI	Primary key
notice_id	BIGINT FK NN	Owning notice. Deleted with it (`orphanRemoval`)
target_type	VARCHAR(20) NN	COLLEGE, ROLE, DEPARTMENT, ROLE_DEPARTMENT, SEMESTER, CLASS, SPECIFIC_USER
target_role	VARCHAR(20)	Role filter; null means any role
department_id	BIGINT FK	Department filter; null means any
semester	INT	Semester filter; null means any
class_name	VARCHAR(20)	Class filter; null means any
user_id	BIGINT FK	Specific user; null means any

10.7 attachments

Column	Type	Meaning
id	BIGINT PK AI	Primary key
notice_id	BIGINT FK NN	Owning notice
original_file_name	VARCHAR(255) NN	Sanitised name (path parts removed, odd characters replaced)
cloudinary_public_id	VARCHAR(255) NN	Cloudinary asset id; needed to sign downloads and to delete
cloudinary_url	VARCHAR(600) NN	Stored secure_url. **Not exposed** by the API and not usable without a signature
resource_type	VARCHAR(20) NN	image (jpg, jpeg, png) or raw (pdf, doc, docx)
file_format	VARCHAR(10) NN	Extension
file_size	BIGINT NN	Bytes
uploaded_by, uploaded_at	BIGINT FK / DATETIME NN	Uploader and time

10.8 notifications

Index: idx_notification_user (user_id).

Column	Type	Meaning
id	BIGINT PK AI	Primary key
user_id	BIGINT FK NN	Recipient
notice_id	BIGINT FK NULL	Related notice, or null for non-notice notifications
message	VARCHAR(500) NN	Text shown in the list
link	VARCHAR(200)	In-app path for non-notice notifications, e.g. /account-requests
is_read	BOOLEAN NN	Read flag
created_at	DATETIME NN	Time

10.9 notice_reads, notice_acknowledgements, saved_notices

Each table has a unique constraint on (notice_id, user_id) (uk_read_notice_user, uk_ack_notice_user, uk_saved_notice_user), so a person can read, acknowledge or save a notice only once. saved_notices also has idx_saved_user (user_id).

Column	Type	Meaning
id	BIGINT PK AI	Primary key (all three tables)
notice_id	BIGINT FK NN	The notice
user_id	BIGINT FK NN	The user
read_at / acknowledged_at / saved_at	DATETIME NN	One time column per table

10.10 audit_logs

Column	Type	Meaning
id	BIGINT PK AI	Primary key
user_id	BIGINT	Actor. **Not a foreign key**, so history survives; null means the system (scheduler)
user_email	VARCHAR(160)	Actor's email at the time, or system
action	VARCHAR(40) NN	One of 27 AuditAction values
entity_type	VARCHAR(40) NN	Notice, User, Attachment, Department, Category, ClassTeacher
entity_id	BIGINT	Id of the affected row
description	VARCHAR(1000)	Human-readable detail (truncated if longer)
event_time	DATETIME NN	When. Indexed by idx_audit_time

10.11 Relationships

```
departments 1 ──< users                (a department has many users)
departments 1 ──< class_teachers >── 1 users (teacher)
users       1 ──< notices              (created_by)
users       1 ──< notices              (reviewed_by)
departments 1 ──< notices              (author's department)
notice_categories 1 ──< notices
notices     1 ──< notice_targets >── 0..1 departments, 0..1 users
notices     1 ──< attachments >── 1 users (uploaded_by)
notices     1 ──< notifications >── 1 users
notices     1 ──< notice_reads | notice_acknowledgements | saved_notices >── 1 users
audit_logs  (no foreign keys, by design)
```

Three design decisions worth defending in a viva

Rows are never hard-deleted for people. Users are deactivated, not removed, so notices, audit entries and statistics keep working. Only notices (with their dependants) can be deleted.

The audience is data, not code. Because targets are rows with nullable columns, one matching rule serves every audience type, and a new audience shape needs no schema change.

Old databases upgrade themselves. Columns added later are nullable and have defaults in the entity getters, and two start-up runners back-fill old rows (see §14.9).

§11 API Documentation — all 73 endpoints

Base URL `http://localhost:8080`. Interactive documentation is at `/swagger-ui.html` (click **Authorize** and paste the token from `/api/auth/login`). It is switched off by the production profile.

Conventions that apply to every endpoint

- Identity is **always** taken from the JWT and re-loaded from the database, never from the request body.
- Protected endpoints need the header `Authorization: Bearer <jwt>`. Missing or invalid token → **401**. Valid token, wrong role → **403**.
- `@Valid` failures return **400** with `{ success:false, message:"Validation failed", timestamp, errors:{ field: message } }`. All other errors use the same shape without `errors`.
- Paged responses are `{ content, pageNumber, pageSize, totalElements, totalPages, last }`.
- Page size is clamped to 1–100 by the server (public notices: 1–50).
- `CREATORS` below means `SUPER_ADMIN`, `PRINCIPAL`, `HOD`, `FACULTY`.

11.1 Authentication — /api/auth

Endpoint	Access	Request	Response · errors · flow
`POST /api/auth/register`	Public	role (STUDENT default, FACULTY, HOD), institutionType, fullName, email, password, departmentId, semester, className, phone	201 `RegisterResponse{message,status,waitingFor}`. 400 validation, wrong wing, semester above the programme's count, class missing, department already has an HOD, email already used. 429 rate limit. Flow: check role → department → wing → student fields → HOD vacancy → email unused (a REJECTED account may re-apply) → rate limit → BCrypt hash → save PENDING → audit → notify approvers.
`POST /api/auth/login`	Public	email, password	200 `AuthResponse{token,tokenType,expiresInMs,user}`. **401** same message for unknown email and wrong password. **403** with an `account` object for pending, rejected, suspended or closed accounts (only after a correct password). **429** after repeated failures.
`POST /api/auth/logout`	Public	—	200 `{success:true,"Signed out"}`. Stateless: the client discards its token.
`POST /api/auth/forgot-password`	Public	email	200 with the **same** message whether or not the email exists. Active users get a 30-minute link by email (or logged when mail is off).
`POST /api/auth/reset-password`	Public	token, newPassword	200 on success. 400 if the token is unknown or expired. Clears the token afterwards.

11.2 Users — /api/users

Endpoint	Access	Request	Response · errors · flow
`GET /api/users`	SUPER_ADMIN	q, role, departmentId, active, status, page, size	`PageResponse<UserResponse>` sorted by name.
`GET /api/users/lookup`	CREATORS	q, role, departmentId, semester	Up to 20 active users for 'specific users' targeting. **Scoped by caller:** HOD sees own department, Faculty only students of own department. 403 for students.
`GET /api/users/me`	Signed in	—	Current user's profile.
`PUT /api/users/me`	Signed in	fullName, phone	Updated profile; audited.
`PUT /api/users/me/password`	Signed in	currentPassword, newPassword	200. 400 'Current password is incorrect' or a weak new password.
`GET /api/users/{id}`	SUPER_ADMIN	—	One user. 404 if unknown.
`POST /api/users`	SUPER_ADMIN	fullName, email, password, role, departmentId, semester, className, phone, active	201 `UserResponse`. 400 duplicate email; HOD/Faculty need a department; students need department, semester and class.
`PUT /api/users/{id}`	SUPER_ADMIN	fields above + optional newPassword	Updated user. You cannot change your own role or deactivate yourself. Editing does not change a pending or suspended status.
`DELETE /api/users/{id}`	SUPER_ADMIN	—	**Soft delete**: sets DEACTIVATED. 400 if it is yourself.
`PUT /api/users/{id}/activate`	SUPER_ADMIN	—	Sets APPROVED, clears the reason.
`PUT /api/users/{id}/deactivate`	SUPER_ADMIN	—	Sets DEACTIVATED.
`PUT /api/users/{id}/suspend`	SUPER_ADMIN	reason (required, ≤ 500)	Sets SUSPENDED with the reason, which the person sees at sign-in. 400 if yourself.

11.3 Account requests and class teachers

Endpoint	Access	Request	Response · errors · flow
`GET /api/registrations/pending`	CREATORS	—	Pending requests the caller may review, oldest first, each with `waitingFor` and `primary` (true if waiting specifically for the caller).
`GET /api/registrations/pending/count`	CREATORS	—	`{count}` of requests waiting for the caller specifically (the menu badge).
`POST /api/registrations/{id}/approve`	CREATORS	—	Sets APPROVED; notifies and emails the applicant. 403 if the caller may not review it; 400 if already handled or if a second HOD would be approved.
`POST /api/registrations/{id}/reject`	CREATORS	reason (required)	Sets REJECTED with the reason; emails the applicant.
`GET /api/class-teachers`	SUPER_ADMIN, PRINCIPAL, HOD	departmentId	Assignments with student counts. HOD is pinned to own department.
`GET /api/class-teachers/options`	same	departmentId	Approved teachers of the department and its classes (semester, class, students).
`PUT /api/class-teachers`	same	departmentId, semester 1–12, className, teacherId	Creates or **replaces** the class teacher. 400 unless the teacher is an approved teacher of that department.
`DELETE /api/class-teachers/{id}`	same	—	204. New student requests for that class then go to the HOD.

11.4 Departments and categories

Endpoint	Access	Request	Response · errors · flow
`GET /api/departments`	**Public**	—	All departments sorted by name (needed by the registration form).
`POST /api/departments`	SUPER_ADMIN	name, code, description, institutionType, programName, semesterCount 1–12	201. 400 if the name already exists **in that wing** or the code exists.
`PUT /api/departments/{id}`	SUPER_ADMIN	same	Updated. 400 if moving to another wing while it still has users.
`DELETE /api/departments/{id}`	SUPER_ADMIN	—	204. 400 if it still has users, notices or notice targets.
`GET /api/categories`	Signed in	includeInactive (honoured only for the Super Admin)	Active categories sorted by name.
`POST /api/categories`	SUPER_ADMIN	name ≤ 80, description, active	201. 400 duplicate name.
`PUT /api/categories/{id}`	SUPER_ADMIN	same	Updated.
`DELETE /api/categories/{id}`	SUPER_ADMIN	—	204. 400 'Deactivate it instead' when any notice uses it.

11.5 Notices — /api/notices (1 of 2)

Endpoint	Access	Request	Response · errors · flow
`GET /api/notices`	Signed in	scope = feed, history, mine, approvals, saved, all; q, categoryId, departmentId, semester, priority, status, creatorId, fromDate, toDate, unreadOnly, page, size (default 10)	`PageResponse<NoticeSummaryResponse>`. **feed** = published, live and targeted to me; **history** = published or expired and targeted to me; **mine** = created by me (not for students); **approvals** = HOD only, faculty notices of own department (default PENDING_APPROVAL); **saved** = my list limited to what I can still see; **all** = Super Admin only. Unknown scope → 400.
`GET /api/notices/target-options`	CREATORS	—	`{allowedTypes, allowedRoles, fixedDepartmentId, fixedDepartmentName, requiresApproval}` for the caller's role.
`POST /api/notices/recipient-preview`	CREATORS	targets[]	`{total, byRole, audience[]}`. Runs the **same** validation and authorisation as saving, so it cannot reveal the size of a forbidden audience.
`GET /api/notices/{id}`	Signed in	—	`NoticeDetailResponse` with a `permissions` object. 404 unknown. **403 'You are not authorized to view this notice'** if the caller may not see it. Opening a published notice you are addressed by records a read.
`POST /api/notices`	CREATORS	title, description, categoryId, priority, publishDate, expiryDate (@Future), acknowledgementRequired, targets[] (≤ 50), visibility, institutionScope	201, **always DRAFT**. 400 no target on a private notice, no wing on a public notice, inactive category. 403 forbidden audience, or a public wing other than your own (HOD/Faculty).
`PUT /api/notices/{id}`	CREATORS	same	Only the creator, and only in an editable state (Faculty: DRAFT or REJECTED; others: DRAFT, SCHEDULED or PUBLISHED). Editing a REJECTED notice returns it to DRAFT.
`DELETE /api/notices/{id}`	CREATORS	—	204. Super Admin any; creator for own (Faculty: only DRAFT or REJECTED). Deletes Cloudinary files **first**, then notifications, reads, acknowledgements, saved entries and the notice.

11.6 Notices — lifecycle and engagement (2 of 2)

Endpoint	Access	Request	Response · errors · flow
`POST /api/notices/{id}/publish`	SUPER_ADMIN, PRINCIPAL, HOD	—	Creator's own DRAFT only. Future publish date → SCHEDULED; otherwise PUBLISHED and recipients notified. 403 for Faculty ('must be submitted for HOD approval'). 400 if the expiry has passed.
`POST /api/notices/{id}/submit`	FACULTY	—	DRAFT or REJECTED → PENDING_APPROVAL; clears the earlier rejection; notifies the department's HODs. 400 if no department.
`POST /api/notices/{id}/approve`	HOD	—	HOD of the author's department, notice PENDING_APPROVAL. Records reviewer, then publishes or schedules, and notifies the author.
`POST /api/notices/{id}/reject`	HOD	reason (required, ≤ 1000)	→ REJECTED with the reason; the author is notified. Audit text carries the reason for the timeline.
`POST /api/notices/{id}/archive`	CREATORS	—	Super Admin or creator; notice must be PUBLISHED or EXPIRED.
`POST /api/notices/{id}/acknowledge`	Signed in	—	Recipients only, notice PUBLISHED and requiring acknowledgement. Also records a read. Idempotent.
`PUT /api/notices/{id}/save`	Signed in	—	Adds to the reading list if the caller can keep seeing it. Saving twice is harmless.
`DELETE /api/notices/{id}/save`	Signed in	—	204. Works even if the notice is no longer visible.
`POST /api/notices/{id}/read`	Signed in	—	Marks read for a recipient who is not the creator.
`GET /api/notices/{id}/stats`	CREATORS	—	`{totalRecipients, read, notRead, acknowledged, notAcknowledged}` over **intended recipients only**. Creator, reviewing HOD or Super Admin.
`GET /api/notices/{id}/timeline`	CREATORS	—	Lifecycle events built from the audit log; same people as stats. Repeated edits are merged and counted.

11.7 Attachments, notifications, dashboards, audit, AI

Endpoint	Access	Request	Response · errors · flow
`POST /api/notices/{id}/attachments`	CREATORS	multipart `files`	201 list of `AttachmentResponse`. Needs permission to edit the notice. 400 more than 10 attachments, bad type, size, MIME or signature. 502 Cloudinary failure (files already pushed are removed).
`GET /api/notices/{id}/attachments`	Signed in	—	Attachment metadata; 403 if the caller cannot view the notice.
`GET /api/attachments/{id}/download`	Signed in	—	`{url, fileName, expiresAt}` — a **signed link valid for 5 minutes**, issued only after the visibility check.
`DELETE /api/attachments/{id}`	CREATORS	—	204. Editor of the notice or Super Admin. Cloud file first, then the row.
`GET /api/notifications`	Signed in	unreadOnly, page, size	Own notifications, newest first.
`GET /api/notifications/unread-count`	Signed in	—	`{count}`.
`PUT /api/notifications/{id}/read`	Signed in	—	Marks one read. Someone else's id looks like 404.
`PUT /api/notifications/read-all`	Signed in	—	`{success, "N notification(s) marked as read"}`.
`DELETE /api/notifications/{id}`	Signed in	—	204; owner only.
`GET /api/dashboard/admin | principal | hod | faculty | student`	The matching role only	—	`{role, counts{}, breakdown{}, recentNotices[], readStatistics[]}`. Another role gets 403.
`GET /api/audit-logs`	SUPER_ADMIN	action, userId, entityType, page, size	Newest first.
`POST /api/ai/notice-draft`	CREATORS	text (5–4000 chars)	201 `{source:"AI"|"MOCK", suggestion, draft}`. The notice is saved as a **DRAFT** with `aiGenerated=true`.

11.8 Public site — /api/public (no sign-in)

Endpoint	Access	Request	Response · errors · flow
`GET /api/public/college`	Public	—	Name, tagline, about, address, phone, email, admission phone, office hours.
`GET /api/public/institutions`	Public	—	Engineering and Polytechnic with programmes and departments, from the department table.
`GET /api/public/categories`	Public	—	Active categories (id, name).
`GET /api/public/notices`	Public	institution, categoryId, q, priority, fromDate, toDate, page, size (default 12)	Only notices that are **public, published, already live and not expired**. `institution=ENGINEERING` also includes notices for BOTH.
`GET /api/public/notices/{id}`	Public	—	`PublicNoticeDetail` (no creator id, targets, status or review data). **404** for anything not publicly visible, exactly like an id that does not exist.
`GET /api/public/notices/{id}/attachments/{attachmentId}/link`	Public	—	5-minute signed link. 404 unless the notice is visible **and** the attachment belongs to that notice.

11.9 Example: sign in, then read the feed

```
POST /api/auth/login
{ "email": "student.cs6a@college.edu", "password": "Password@123" }

200 OK
{ "token": "eyJhbGciOiJIUzUxMiJ9...", "tokenType": "Bearer", "expiresInMs": 86400000,
  "user": { "id": 7, "fullName": "Student One", "role": "STUDENT", "departmentName": "Computer Engineering"
,
            "semester": 6, "className": "A", "status": "APPROVED", ... } }

GET /api/notices?scope=feed&page=0&size=10
Authorization: Bearer eyJhbGciOiJIUzUxMiJ9...

200 OK   { "content": [ {"id": 12, "title": "Semester 6 end-semester examination timetable", ... } ],
           "pageNumber": 0, "pageSize": 10, "totalElements": 5, "totalPages": 1, "last": true }

GET /api/notices/8     (the HOD-only meeting notice)
403 Forbidden  { "success": false, "message": "You are not authorized to view this notice", ... }
```

§12 Authentication — login to protected access

Authentication answers "who are you?"; authorisation answers "what may you do?". This project separates them cleanly: a signed token proves identity, and the **database** is asked about role, status and department on every request.

12.1 Registration to first sign-in

1. The visitor opens `/register`, chooses a role (Student, Teacher, HOD), then the **institution** (Engineering or Polytechnic), then a department of that wing, then semester and class if a student.
2. `POST /api/auth/register`. The server validates, checks the department belongs to the chosen wing, checks the semester does not exceed the programme (8 for B.Tech, 6 for Diploma), checks an HOD does not already exist for the department, and applies the registration rate limit.
3. The password is hashed with **BCrypt** and the account is saved as `PENDING`. The applicant receives no token.
4. `RegistrationService.route()` picks the approver, who gets an in-app notification with a link to `/account-requests`.
5. The approver approves; the applicant is notified (and emailed if mail is on). Only now can they sign in.

> **Development shortcut**
> `app.auth.auto-activate-registrations=true` approves **students** immediately. It never applies to teachers or HODs. It defaults to `false`; the integration tests switch it on so the suite can sign up users without a second actor.

12.2 Sign-in, step by step

1. `POST /api/auth/login` with email and password. The email is trimmed and lower-cased.
2. `AuthRateLimiter.checkLogin` throws **429** if this email+address pair has 5 recent failures, or this address has 20.
3. The user is looked up. If missing, a BCrypt comparison against a throw-away **dummy hash** is still done so that the response time does not reveal whether the email exists.
4. A wrong password records a failure and returns **401 "Invalid email or password"**, identical for unknown emails and wrong passwords.
5. A correct password clears the email+address counter.
6. Only now is the account status checked. If it is not `APPROVED`, the server returns **403** with an `account` object (status, role, department, who it is waiting for, reason, history timeline) and **no token**.
7. Otherwise `JwtService.generateToken` issues the token and the user object is returned.

12.3 The token

Property	Value
Algorithm	HS512 (HMAC-SHA-512)
Subject (`sub`)	The user's email
Custom claims	`uid` (user id) and `role`
Issued / expiry	`iat`, and `exp` = 24 hours (`app.jwt.expiration-ms=86400000`)
Secret	`app.jwt.secret` / `JWT_SECRET`. **Must be at least 64 bytes** or the application refuses to start
Where the client stores it	`localStorage` key `cnb_token`
How it is sent	`Authorization: Bearer <token>` added by an Axios interceptor

12.4 What happens on every protected request

```
Authorization: Bearer <jwt>
   -> JwtAuthenticationFilter
        verify signature and expiry          (invalid -> stay anonymous -> 401)
        load the user by email FROM THE DB   (deleted user -> anonymous)
        user.isEnabled()?  (status APPROVED) (no -> anonymous -> 401)
        set SecurityContext = UserPrincipal(id, email, role)
   -> URL rules in SecurityConfig            (public vs authenticated)
   -> @PreAuthorize on the controller method (role check -> 403)
   -> CurrentUserService.currentUser()       (fresh entity: role, dept, semester, class)
   -> service-level checks                   (TargetPolicy, NoticeAccessService)
```

> **Why the user is reloaded on every request**
> A JWT is valid until it expires. If the server trusted the token alone, a user who was suspended or whose department changed would keep their old powers for up to 24 hours. Reloading the account on every request makes suspension, closure and role changes take effect **immediately**. The cost is one indexed primary-key or unique-email lookup per request. A test (`suspendedUserIsToldWhyAndLosesTokenAccess`) proves the behaviour.

12.5 Sign-out and expiry

Sign-out is **stateless**: `POST /api/auth/logout` returns a message and the browser deletes its token. The server keeps no list of tokens, so a copied token stays valid until it expires or the account is blocked (see §19). When any API call returns 401 *while a token is stored*, the Axios interceptor clears the token and sends the user to `/login?expired=1`, where the page says "Your session ended. Sign in again."

12.6 Password reset

1. `POST /api/auth/forgot-password` returns the same message for every email.
2. For an **active** user, 32 random bytes from `SecureRandom` become a URL-safe token. The database stores only `SHA-256(token)` and an expiry 30 minutes ahead.
3. The link `FRONTEND_URL/reset-password?token=...` is emailed. If mail is disabled it is written to the server log, which is how a developer tests the flow.
4. `POST /api/auth/reset-password` hashes the supplied token, finds the user, checks expiry, applies the new (strength-checked) password, and clears the token so it cannot be reused.

§13 Application Workflow — what happens on startup and after

13.1 Backend start-up sequence

1. `NoticeBoardApplication` starts; `@PostConstruct` sets the JVM default time zone to `app.timezone` (`Asia/Kolkata`) so all `LocalDateTime` values agree.
2. `JwtService` is constructed and **fails fast** if the secret is shorter than 64 bytes.
3. The datasource connects. The URL contains `createDatabaseIfNotExist=true`, so the schema is created if missing. Hibernate runs `ddl-auto=update`, creating or altering the 12 tables.
4. `AccountStatusBackfill` (`@Order(0)`) gives a status to old users: active → APPROVED, inactive → DEACTIVATED.
5. `InstitutionUpgrade` (`@Order(0)`) sets old departments to ENGINEERING and old notices to PRIVATE, and on MySQL drops the old unique index on `departments.name`.
6. `DataSeeder` runs **only** if `app.seed.enabled=true` and the `users` table is empty.
7. The embedded Tomcat listens on port 8080. Ten seconds later the first `NoticeScheduler` run happens, and then one run a minute after the previous one finishes.

13.2 Frontend start-up sequence

1. The browser loads `index.html`; the inline script applies the saved theme before first paint.
2. `main.jsx` mounts the providers. `AuthProvider` finds a stored token and calls `GET /users/me`; until it answers, `RootFrame` shows a loading indicator.
3. If that call fails (expired or blocked), the token is cleared and the visitor sees the public site.
4. If it succeeds, `RootFrame` renders the portal `Layout` and `Home` renders the role's dashboard, which loads `/dashboard/<role>` and the urgent banner.
5. `Layout` starts a 60-second poll for the unread-notification count and, for approvers, the pending-request count.

13.3 Three end-to-end stories

Story A — a new student joins

1. A new student registers: Engineering, Computer Engineering, semester 6, class A. The account is `PENDING`.
2. `classTeacher(CS, 6, A)` finds that class's teacher, so the request waits for them. They see a badge (1) and an in-app notification.
3. The student signs in and sees "waiting for approval by the class teacher of Computer Engineering semester 6 class A" with a timeline.
4. The class teacher approves. The student receives a welcome notification and can sign in; they now match every notice targeted at CS, semester 6 or class A.

Story B — a faculty notice reaches a class

1. The class teacher writes "Class 6A lab assignment", audience **CS · semester 6 · class A**. The audience panel shows "1 student will receive this notice".

2. They choose *Save and submit for approval*. Status becomes `PENDING_APPROVAL`; the HOD is notified.
3. The HOD opens *Notice approvals*, reads it and approves. The same transaction publishes it, notifies the class and tells the author.
4. Students in 6A see it in **Notices**; students in 6B and other departments do not, and a direct URL gives 403.

Story C — an urgent circular

1. The Principal writes an Urgent notice for the entire college and publishes it.
2. Every active user (except the Principal) gets an in-app notification **and** an email, sent in the background.
3. Each dashboard shows the red urgent banner until the notice expires. Each recipient is asked to confirm "I have read this notice".
4. The Principal's dashboard shows 'N of M read · K acknowledged', counted over intended recipients.

§14 Business Logic — all major workflows

14.1 Targeting and the matching rule

A notice has one or more `NoticeTarget` rows. The **type** documents intent and decides which fields are required; the **matching** is always the same.

> **The rule**
> A target row matches a user when **every non-null field** in the row equals the user's value: `targetRole`, `department`, `semester`, `className`, `user`. A notice is visible to a user if **any** row matches. Nulls are wildcards.

Target type	Fields filled by `normalise()`	Example
`COLLEGE`	none (all wildcards)	Everyone
`ROLE`	role	All HODs
`DEPARTMENT`	department	Everyone in Mechanical
`ROLE_DEPARTMENT`	role + department	CS faculty
`SEMESTER`	role (default STUDENT) + department + semester	CS semester 6 students
`CLASS`	role (default STUDENT) + department + semester + class	CS 6A students
`SPECIFIC_USER`	user	One named student

Notice	Target rows	Who sees it
Principal → HODs	`ROLE, HOD`	All HODs only
Principal → HODs + faculty	`ROLE, HOD` + `ROLE, FACULTY`	HODs and faculty, not students
Principal → everyone	`COLLEGE`	Everyone
HOD → CS faculty	`ROLE_DEPARTMENT, FACULTY, CS`	CS faculty
HOD → CS sem 6	`SEMESTER, STUDENT, CS, 6`	CS semester 6 students
Faculty → 6A	`CLASS, STUDENT, CS, 6, A`	CS 6A students, once the HOD approves

Who may address whom (`TargetPolicy.authorise`)

Role	Allowed target types	Allowed roles	Restriction
Super Admin, Principal	all seven	any	None
HOD	DEPARTMENT, ROLE_DEPARTMENT, SEMESTER, CLASS, SPECIFIC_USER	Faculty, Student	Own department only. COLLEGE and ROLE are forbidden. Specific users must be from the own department.
Faculty	ROLE_DEPARTMENT, SEMESTER, CLASS, SPECIFIC_USER	Student	Own department's **students** only. Requires approval.
Student	none	none	Cannot create notices (403)

Selected specific users must be **active**. Duplicate rows are collapsed using a key of all six fields. The browser's `TargetBuilder` offers only what this table allows (from `/notices/target-options`), but the server enforces it regardless.

14.2 Account approval routing

```
HOD      -> Principal                                  (no Principal -> "the administrator")
Teacher  -> HOD of the department                      (no HOD       -> Principal)
Student  -> class teacher of dept + semester + class   (none         -> department HOD -> Princip
al)
```

Reviewer	May review
Super Admin, Principal	Anyone
HOD	Teachers and students of **their own department** only. A department can have only **one** approved HOD.
Faculty	Only students of the class they are class teacher of
Student	No one

Each queue has a `primary` flag: true when the request is waiting **for that reviewer specifically**. The menu badge counts primary requests only. The Principal's queue shows HODs and anything escalated to them (not every student). The applicant never chooses the approver. Because every approval is scoped to the applicant's department, an Engineering HOD can never approve a Polytechnic account or notice, and the reverse.

Account lifecycle: `PENDING → APPROVED`; `PENDING → REJECTED` (reason; the person may apply again with the same email); `APPROVED → SUSPENDED` (reason; lifted by the Super Admin); `APPROVED → DEACTIVATED` (closed; can be reopened). A student, teacher or HOD can reapply while `REJECTED`; any other existing account blocks the email.

14.3 Notice lifecycle

```
Principal / HOD / Admin:
  DRAFT --publish--> PUBLISHED --expiry--> EXPIRED --archive--> ARCHIVED
    |                    ^
    '--(future date)--> SCHEDULED --scheduler (publish date reached)--'

Faculty:
  DRAFT --submit--> PENDING_APPROVAL --HOD approve--> (APPROVED) -> PUBLISHED or SCHEDULED
                         '--HOD reject (reason)--> REJECTED --edit--> DRAFT --submit--> ...
```

Status	Meaning	Visible to
`DRAFT`	Being written; every new notice starts here, including AI drafts	Creator and Super Admin
`PENDING_APPROVAL`	Faculty notice waiting for the department HOD	Creator, that HOD, Super Admin
`APPROVED`	Transient: approval recorded, then immediately published or scheduled	—
`REJECTED`	Returned with a reason; editing turns it back into a draft	Creator, that HOD, Super Admin
`SCHEDULED`	Approved or published with a future publish date	Creator, Super Admin
`PUBLISHED`	Live (until its expiry) and visible to recipients; recipients notified once	Recipients, creator, Super Admin; visitors if public
`EXPIRED`	Past expiry; stays in recipients' history	Recipients (history), creator, Super Admin

Status	Meaning	Visible to
`ARCHIVED`	Hidden from recipients' feed and history by the creator or Super Admin	Creator, Super Admin

Date rules: expiry must be in the future; expiry must be after the publish date; a notice whose expiry has passed cannot be published, submitted or approved. For a published notice the publish date cannot be changed. A scheduled notice edited so that its publish time is no longer in the future is published immediately.

The scheduler (`processDueNotices`, every minute): (1) every `SCHEDULED` notice whose publish date has arrived is published, unless its expiry has already passed, in which case it becomes `EXPIRED` with the audit note "Expired before it was published"; (2) every `PUBLISHED` notice whose expiry has arrived becomes `EXPIRED`. Both write system audit entries (user id null).

14.4 Faculty approval workflow

- **Submit**: only the author, only from `DRAFT` or `REJECTED`, only if the author has a department. Clears any earlier rejection, sets `PENDING_APPROVAL`, notifies the department's HODs: "Approval needed: <title> by <name>".
- **Approve**: only the HOD of the department the notice was created in. Records `reviewedBy` and `reviewedAt`, sets `APPROVED`, then publishes or schedules in the same transaction, then tells the author.
- **Reject**: needs a reason (max 1,000). The reason is stored on the notice, written into the audit entry, and shown to the author in a red box and in their notification.
- **Resubmit**: the author edits the rejected notice (which becomes a draft) and submits again.
- A faculty notice is editable only while `DRAFT` or `REJECTED`; once submitted it is locked.

14.5 Public notices and who may publish them

A notice is **private** (to members, through targets) or **public** with a wing: Engineering, Polytechnic or both. A public notice may *also* have targets; with no targets, nobody is notified.

Role	Public notices
Super Admin, Principal	Any wing, published directly
HOD	Own department's wing only, published directly
Faculty	Own wing only, and only after HOD approval
Student	Cannot create notices

A notice is visible to visitors **only while it is public, published, already live and not past its expiry**. This is defined once in Java (`Notice.isPubliclyVisible`) and once in SQL (`NoticeSpecifications.publiclyVisible`); the comment on the SQL version says to keep the two in step. Drafts, pending, rejected, scheduled, expired, archived and private notices all answer **404** on the public API.

14.6 Attachments

1. `canEdit` must be true for the caller. At most 10 attachments per notice.
2. **Validate everything before uploading anything**: extension whitelist, MIME type for that extension, size ≤ 10 MB, and the first bytes of the file (`%PDF`, PNG header, `FF D8 FF`, OLE2 header, ZIP header).
3. Each file is uploaded to Cloudinary under `college-notice-board/notices/<noticeId>/` with a random UUID as its public id (raw files keep their extension) and delivery type `authenticated`.

4. The metadata row is saved and an audit entry written. If any file fails, files already uploaded in this request are deleted from Cloudinary (**compensation**).
5. Download: `canView` is checked, then a link signed with an expiry 5 minutes ahead is returned.
6. Delete: Cloudinary first; if that fails the row is kept and the user gets 502, so no file is left without a reference. Deleting a notice does this for every attachment before deleting the notice.

14.7 Notifications, email and read statistics

- **On publish** (immediately, or from the scheduler): `RecipientService` resolves active users matching any target, excluding the creator. One `Notification` per recipient: "New notice: <title>", or "URGENT notice: <title>" for non-normal priority.
- **Email** only when the priority is IMPORTANT or URGENT: subject `[URGENT] <title>`, body with the text and a link; one separate message per recipient, sent asynchronously.
- **Other notifications**: approval needed, approved, rejected, new account request, account approved.
- **Read** is recorded the first time a recipient opens the published notice (the creator is excluded), and also marks that notice's notifications read. **Acknowledge** is a separate explicit action.
- **Statistics**: the set of recipient ids is computed, then read and acknowledged ids are intersected with it. `notAcknowledged` is zero for notices that do not need acknowledgement.

14.8 AI notice assistant

1. A creator types a sentence, e.g. "College will remain closed tomorrow because of a public holiday."
2. `AnthropicAiSuggestionClient` is used if `AI_API_KEY` is set. It sends the list of active categories and the current time, and asks for **only** a JSON object. Fences are stripped, enums parsed leniently, dates parsed tolerantly. Any failure returns empty.
3. Otherwise `MockAiSuggestionClient` is used: keyword rules choose the category (Holiday, Examination, Placement, ...), priority (words like *urgent*, *mandatory*, *tomorrow*), audience (HOD, faculty, students), and a title from the first eight words.
4. `sanitise` guarantees a title ≤ 200 chars, uses the user's text if the model left the body empty, drops past publish dates, and sets expiry to 7 days if the suggestion is missing or implausible.
5. The suggested audience is used **only if the creator's role is allowed to use it** (`TargetPolicy`); otherwise the role's default audience is used. Specific users are never suggested by AI.
6. The notice is saved as a **DRAFT** with `aiGenerated=true`, and the response says whether the source was `AI` or `MOCK`.

> **Safety principle**
> AI output is **never** published automatically. It is data that the creator must review and push through the normal publish or approval workflow. A test (`aiGeneratedNoticeIsAlwaysADraft`) enforces this.

14.9 Upgrading older databases

Features were added in stages, so existing databases must keep working. Hibernate adds new nullable columns, entity getters supply defaults, and two idempotent runners fix the data: `AccountStatusBackfill` sets a status on every old user from their `active` flag, and `InstitutionUpgrade` marks old departments as Engineering and old notices as private, then drops the obsolete unique index on `departments.name` (MySQL only), because names are now unique **per wing**. Each step does nothing once applied.

§15 Security Features

Threat	Defence in this project	Where
Reading a notice by guessing its id (IDOR)	`NoticeAccessService.assertCanView` on detail, attachments, download, stats and timeline; 403 on a direct URL	service layer
Hiding things only in the UI	Visibility is enforced by SQL (`targetedTo`) and Java (`TargetMatcher`); the UI only mirrors it	`NoticeSpecifications`
Privilege escalation by choosing a bigger audience	`TargetPolicy.authorise` per role; the recipient preview uses the same checks	`TargetPolicy`
Self-chosen approver or role	Registration allows only Student, Teacher, HOD; the system picks the approver	`AuthService`, `RegistrationService`
Stolen password database	BCrypt hashes; reset tokens stored as SHA-256	`SecurityConfig`, `AuthService`
Password guessing	5 failures per email+address and 20 per address in 15 minutes → 429	`AuthRateLimiter`
Discovering which emails have accounts	Same 401 for unknown email and wrong password; dummy-hash timing; status only after a correct password; forgot-password always the same message	`AuthService`
Registration floods	10 registrations per address per hour	`AuthRateLimiter`
Forged or edited token	HS512 signature verified; secret ≥ 64 bytes enforced at start-up	`JwtService`
Use of a token after suspension	Account reloaded and checked on every request	`JwtAuthenticationFilter`
Malicious uploads	Extension + MIME + size + magic-byte check; sanitised file name; random storage name	`FileValidator`
Direct access to stored files	Cloudinary `authenticated` delivery; 5-minute signed links after a visibility check	`CloudinaryService`, `AttachmentService`
Public site leaking private data	Separate DTOs; every query starts from `publiclyVisible`; non-public → 404	`PublicNoticeService`
Cross-origin abuse	CORS allows only configured origins, 6 methods, 2 headers	`SecurityConfig`
SQL injection	Spring Data and Criteria API with bound parameters; `%` and `_` are escaped in search text	repositories
Secrets in the repository	`.env.example` has placeholders; production reads environment variables; Swagger off in production	`application-prod.properties`
Tampering with history	Users are deactivated, never deleted; the audit log has no foreign keys and is Admin-readable only	`AuditService`

Why CSRF protection is disabled

CSRF attacks work because browsers attach **cookies** automatically. This API uses no cookies: the token is kept in `localStorage` and added to an `Authorization` header by JavaScript, which a cross-site form cannot do. If the token were ever moved into a cookie, CSRF protection would have to be switched on again.

> **Honest limits**
> Logout does not revoke a token; the rate limiter is in memory and uses the direct client address; the token is in `localStorage`, which a cross-site-scripting bug could read. These are documented in §19, not hidden.

§16 Error Handling

`GlobalExceptionHandler` (a `@RestControllerAdvice`) converts every failure into one JSON shape: `{ success:false, message, timestamp, errors? }` where `errors` appears only for validation failures. Raw stack traces never reach the client.

Exception	HTTP status	Message source
`MethodArgumentNotValidException`	400	"Validation failed" + field map
`ConstraintViolationException`	400	"Validation failed" + property map
`HttpMessageNotReadableException`, `MethodArgumentTypeMismatchException`	400	"Malformed request: check field names, types and enum values"
`BadRequestException`, `NoticeApprovalException`	400	Specific business message
`BadCredentialsException`	401	"Invalid email or password"
`AuthenticationException`, `UnauthorizedException`	401	"Authentication required"
`AccountStatusException`	403 + `account` object	e.g. "Your account is waiting for approval by …"
`ForbiddenException`, `AccessDeniedException`, `DisabledException`	403	Specific message or "You do not have permission to perform this action"
`ResourceNotFoundException`, `NoResourceFoundException`	404	"Notice not found", "Endpoint not found"
`HttpRequestMethodNotSupportedException`	405	Standard message
`DataIntegrityViolationException`	409	"The operation conflicts with existing data" (detail logged, not returned)
`MaxUploadSizeExceededException`	413	"File is too large. Maximum size is 10 MB"
`MultipartException`	400	"Invalid file upload request"
`TooManyRequestsException`	429	"Too many failed sign-in attempts. Try again in N minutes."
`CloudinaryUploadException`	502	"Could not upload the file to cloud storage. Try again later"
Any other `Exception`	500	"An unexpected error occurred" (stack trace logged only)

Errors raised inside the **security filter** (before any controller) use `JsonSecurityHandlers`, so a 401 or 403 from the filter chain has the same JSON shape. On the client, `errorMessage()` turns the response into one sentence for a toast or an alert, `fieldErrors()` marks individual inputs, and `ErrorState` offers a *Try again* button for failed page loads.

§17 Validation

Validation happens in three places, each with a different job: the **browser** for speed, **Bean Validation** for shape, and the **service layer** for rules that depend on who is asking or on stored data.

Field / input	Bean Validation rule	Service-layer rule
Password	`PASSWORD_REGEX`: 8–64 chars, uppercase, lowercase, digit, symbol	Current password must match when changing
Email	`@NotBlank @Email`, ≤ 160	Unique, case-insensitive; trimmed and lower-cased
Full name	`@NotBlank`, ≤ 120	—
Phone	Register: `^$\|^[0-9+\- ]{7,20}$`; profile: ≤ 20	—
Semester	`@Min(1) @Max(12)`	Students: required, and not above the department's `semesterCount`
Class name	≤ 20	Students: required; stored upper-case
Department	Required on registration	Must exist; must belong to the chosen wing; one HOD per department
Notice title	`@NotBlank`, ≤ 200	Trimmed
Notice text	`@NotBlank`, ≤ 20,000	Trimmed
Category / priority	`@NotNull`	Category must exist and be **active**
Expiry date	`@Future`	After the publish date; not passed when publishing or approving
Targets	≤ 50 rows, each with a type	Per-type required fields; per-role authorisation; users must be active
Public notice	—	Must choose a wing; HOD and Faculty only their own wing
Rejection or suspension reason	`@NotBlank`, ≤ 500 (notice rejection ≤ 1000)	—
Attachments	Multipart size limit: 10 MB per file, 50 MB per request	Extension, MIME, signature, count ≤ 10
AI text	`@NotBlank`, 5–4,000 characters	Role must be able to create notices
Class teacher	Semester 1–12, class ≤ 20, teacher id	Teacher must be an approved teacher of that department
Department / category names	≤ 120 / ≤ 80; code ≤ 20	Unique (name per wing, code overall)

§18 Challenges Faced During Development

18.1 Hiding in the UI is not security

Problem. The first idea for "each notice is visible only to its audience" was to filter the list in React. But anyone can call `GET /api/notices/42` directly, or change the id in the address bar, and read a notice meant for someone else.

Solution. Visibility moved into the backend. Every notice has `NoticeTarget` rows and one matching rule. `NoticeAccessService` is the single gate that **every** read path goes through (list, detail, attachment list, download link, statistics, timeline, saved list), and the detail endpoint returns 403 for a notice the caller is not addressed by.

Lesson. Authorisation belongs where the data is. The UI is a convenience, and the `permissions` object returned with each notice is how the UI learns what to show, rather than guessing.

18.2 One rule, two languages

Problem. The visibility rule had to run in two places: in Java, to answer "may this one user open this one notice?", and in SQL, to answer "which of 10,000 notices are for me?" without loading them all. Two implementations can quietly drift apart.

Solution. The rule was written in the simplest possible form (all non-null fields equal; any row matches) and implemented as `TargetMatcher` and as `NoticeSpecifications.targetedTo` / `usersMatching`. The Javadoc of each points at the other. `TargetMatcherTest` and the integration tests assert that list, detail and recipient counts agree.

Lesson. If a rule must exist twice, make it small enough to state in one sentence and test both copies against the same cases.

18.3 "No targets" must mean nobody

Problem. Public notices may have no targets, because they are meant for the website, not for member notifications. A recipient query built by OR-ing target conditions over an **empty** list can easily degrade into "match everything", which would notify every user about a notice for visitors.

Solution. `RecipientService.resolve` returns an empty list immediately when there are no targets, with a comment explaining why. A public notice with no targets was added to the integration tests: nobody is notified.

Lesson. Empty-set edge cases are where access-control bugs hide. Decide the meaning of "empty" explicitly.

18.4 Who approves the approver?

Problem. A student's account must be approved by the class teacher. But a class may have no teacher, a department may have no HOD yet, and the HOD themself needs approval from someone.

Solution. A fixed chain with fallbacks: student → class teacher → HOD → Principal → administrator; teacher → HOD → Principal; HOD → Principal. Anyone **higher** in the chain may also act. A `primary` flag separates "waiting for me" from "I could step in" so the badge number stays meaningful. One HOD per department avoids two competing approvers.

Lesson. A workflow with no fallback eventually stalls. Design the empty case, not just the happy path.

18.5 Leaking which accounts exist

Problem. A friendly sign-in form that says "no such user" or "your account is pending" lets anyone test whether a given email is registered, and a different response time gives the same information.

Solution. Unknown email and wrong password return the identical 401; a dummy BCrypt comparison keeps timing similar; the account's status is revealed **only after a correct password**; `forgot-password` always returns the same sentence. Failed attempts are counted per email+address and per address.

Lesson. Usability and privacy pull in opposite directions at the login form. Reveal state only to someone who has proved who they are.

18.6 Orphaned files in cloud storage

Problem. Attachments live in Cloudinary, metadata in MySQL. Two systems cannot share one transaction, so a crash or failed upload midway can leave a file with no row, or a row pointing at nothing.

Solution. Validate **all** files before uploading **any**; on failure delete what was already pushed (compensation); on delete, remove the cloud file **first** and keep the row if that fails, returning 502; treat Cloudinary's "not found" as success so a retry is safe.

Lesson. When you cannot have a transaction across two systems, order the steps so the failure that remains is the harmless one (a retryable error rather than a silent leak).

18.7 Read statistics that flatter

Problem. If "read" counts every person who opened a notice, then a stray reader, or the creator previewing their own notice, inflates the percentage and the Principal wrongly believes everyone has been informed.

Solution. Statistics intersect the set of readers with the set of **intended recipients** (creator excluded), and reads are only recorded for recipients. Acknowledgement is a separate, explicit action.

Lesson. A metric is only as honest as its denominator.

18.8 Growing a database that already has data

Problem. Account statuses, the Engineering/Polytechnic split and public notices were added after the first version. Existing rows had null in the new columns, and the old unique index on `departments.name` blocked having "Computer Engineering" in both wings.

Solution. New columns are nullable with defaults in the entity getters (`getStatus`, `getInstitutionType`, `getVisibility`). Two idempotent `ApplicationRunner`s back-fill the data and drop the old index on MySQL by looking it up in `information_schema`.

Lesson. `ddl-auto=update` adds columns but never fixes data or drops constraints. Plan the data migration. (For production, Flyway or Liquibase is the proper tool, see §19.)

18.9 A build that fails on newer JDKs

Problem. Spring Boot 3.3 ships Lombok 1.18.34, which crashes `javac` on JDK 23 and newer with an obscure `TypeTag :: UNKNOWN` error, so the project failed to compile on some team members' machines.

Solution. The Lombok version is pinned to 1.18.42 in `pom.xml`, with a comment explaining why.

Lesson. Pin tool versions that your platform's default does not support, and write the reason next to the pin.

18.10 Slow lists from hidden queries

Problem. A page of 12 notices needed attachment counts, read flags and saved flags for each, which naïvely is 3 × 12 extra queries (the N+1 problem).

Solution. `NoticeService.summaries` fetches those three things for the **whole page** in three batched queries and joins them in memory. Pagination and sorting happen in the database through `Specification` and `Pageable`.

Lesson. Look at the number of queries a page triggers, not just whether it works.

18.11 Losing a half-written notice

Problem. A long notice typed into a form is lost if the tab is closed, the session expires or the laptop sleeps.

Solution. The form auto-saves to `localStorage` 1.5 s after each change, keyed by user and notice, and offers *Restore changes* on return. Nothing reaches the server until the author saves, so half-written text never appears in "My notices" or the audit log.

Lesson. Keep drafts where they cannot pollute shared data, and tell the user plainly what is not restored (attachments must be chosen again).

18.12 Mail must never break publishing

Problem. Emailing 2,000 recipients inside the publish request would make it slow and would make a mail outage fail the publish itself.

Solution. `EmailService.sendToEach` runs on a dedicated thread pool (`@Async("mailExecutor")`), sends one message per recipient so addresses are never shared, catches each failure individually and logs it. Only Important and Urgent notices are emailed at all.

Lesson. Anything outside your control (networks, third parties) belongs off the request path.

§19 Possible Improvements

These are limits of the **current** implementation, stated honestly. Each is a known trade-off for a project of this scope, and each has a clear fix.

Area	Current behaviour	Improvement
Token revocation	Stateless logout. A copied token works until it expires (24 h) or the account is blocked.	Short-lived access token (about 15 min) plus a revocable refresh token stored server-side.
Token storage	JWT in `localStorage`, readable by any script that runs on the page.	`httpOnly`, `Secure`, `SameSite` cookie (then re-enable CSRF protection).
Rate limiter	In memory, single instance; uses the direct remote address, so behind a proxy all users look like one address.	Shared store (Redis) and a trusted `X-Forwarded-For` configuration.
Schema changes	`ddl-auto=update`, plus hand-written upgrade runners.	Flyway or Liquibase migrations.
Admin-created HODs	`UserService.create/update` does not apply the "one HOD per department" rule that registration applies.	Call `ensureHodVacancy` from the admin path too.
Editing a published notice	Allowed for non-faculty creators; changed text is not re-sent and a changed audience is not re-notified.	Notify on material change, or version notices.
Statistics at scale	`statsFor` loads the full recipient id set into memory.	Count with SQL joins, or maintain counters.
Recipient fan-out	Notifications are inserted in the publish transaction.	Batch inserts or an outbox processed in the background.
Search	`LIKE %q%` on title and text.	MySQL full-text index or a search engine.
Email	Plain-text bodies.	HTML templates with the college branding.
Uploads	Type, size and signature checks only.	Antivirus scanning before storing.
Development credentials	a development database username/password and a development JWT secret are defaults in `application.properties`.	Fail start-up in the `prod` profile if the defaults are in use (the prod profile already requires environment variables).
Notification refresh	Counts are polled every 60 s.	WebSocket or server-sent events.
Frontend tests	None.	Vitest and React Testing Library for forms, plus Playwright for end-to-end flows.

§20 Future Scope

- **Refresh tokens and revocation**, so "sign out everywhere" and instant token invalidation are possible.
- **Real-time delivery** by WebSocket or server-sent events instead of polling.
- **More channels**: SMS and WhatsApp for urgent circulars; browser and mobile push.
- **Mobile app / PWA**: the layout already has a bottom navigation bar; an installable offline-friendly app is the natural next step.
- **Notice templates** for recurring announcements (holiday, exam timetable, fee reminder).
- **Multilingual notices**, for example English with Marathi and Hindi versions, chosen by the reader.
- **Bulk user import from CSV** for the start of an academic year, including promotion of students to the next semester.
- **Read reminders** for unacknowledged urgent notices (a second notification or email after a delay).
- **Virus scanning** of uploads and image thumbnails for attachments.
- **Calendar integration**: export notice dates (exams, events) to iCalendar.
- **Richer analytics**: read rates by department, time to first read, and notices that were never opened.
- **Rich-text editing** and inline images in the notice body.
- **Multi-campus support**: make the college the top-level tenant, so one deployment can serve several campuses.

§21 Expected Viva Questions — 87 questions with answers

87 questions specific to this project. Each has a detailed answer for understanding, a short answer to say out loud, and likely follow-ups so you are never caught off guard by the second question.

A · Project, Requirements and Scope

Q1 What is the College Online Notice Board and what problem does it solve?

DETAILED

It is a full-stack web application, built with Spring Boot, React and MySQL, that replaces the physical notice board. A physical board reaches only people who walk past it, cannot target a class, has no approval trail and no read tracking. Here authorised staff create, schedule, approve and publish notices, each notice is visible only to the audience its author chose, and visitors who are not signed in see a public website.

SHORT

> A role-based digital notice board where every notice reaches exactly the audience chosen, after the right approval, with a record of who read it.

Follow-ups: What are the roles? · What does "exactly the audience" mean technically? · Why not use WhatsApp groups?

Q2 Who are the five roles and how does each get an account?

DETAILED

Students, teachers (stored as FACULTY) and HODs self-register and wait for approval. The Super Admin and the Principal are created by the Super Admin, not by self-registration, because they hold the most power. In development a Super Admin is seeded on first start.

SHORT

> Student, teacher and HOD register and get approved; Principal and Super Admin are created by the Super Admin.

Follow-ups: Why can't a Principal self-register? · Who approves an HOD?

Q3 Why must a new account be approved, and who approves it?

DETAILED

A notice board is only trustworthy if the people on it are real members of the college, and some people can address everyone. So every self-registered account starts as PENDING and cannot sign in. The **system** picks the approver: HOD → Principal; teacher → HOD of their department; student → the class teacher of their class, falling back to the HOD, then the Principal. The applicant never chooses.

SHORT

> To admit only verified members. HODs are approved by the Principal, teachers by their HOD, students by their class teacher, and the system chooses the approver.

Follow-ups: What if the class has no class teacher? · Can an Engineering HOD approve a Polytechnic student?

Q4 What did you deliberately not build?

DETAILED

Real-time push (we poll every 60 seconds), SMS and WhatsApp channels, a mobile app, notice templates, multilingual notices, bulk CSV import, and token revocation. They are listed under Future Scope. We chose to make one workflow correct and secure (targeting, approval, public visibility, attachments) rather than many superficial ones.

SHORT

> Push channels, mobile app, templates, bulk import and token revocation. We prioritised correctness of targeting and approval.

Follow-ups: Which of those would you build first? · Why is polling acceptable here?

Q5 What is the difference between a private and a public notice?

DETAILED

A private notice reaches only the college members matched by its targets. A public notice is also readable by anyone on the public website, but only while it is published, already live and not expired, and it carries a wing: Engineering, Polytechnic or both. A public notice may additionally have targets, to notify members; with no targets, nobody is notified.

SHORT

> Private goes to chosen members; public is also on the open website for a wing while live.

Follow-ups: Who may publish a public notice? · What does a visitor get for a draft notice's URL?

Q6 Why does the system have two wings, Engineering and Polytechnic?

DETAILED

Many colleges run a degree and a diploma programme on one campus. Each department belongs to one wing and has its own programme name and semester count (8 for B.Tech, 6 for Diploma). The wing scopes approvals (an Engineering HOD never sees Polytechnic requests) and public notices, and the registration form offers only that wing's departments and semesters.

SHORT

> Because one campus has both programmes; the wing scopes departments, semesters, approvals and public notices.

Follow-ups: Can the same department name exist in both? · How is the semester limit checked?

Q7 What is the single most important design idea in the project?

DETAILED

That visibility is decided by the server, with one small rule. Each notice has target rows; a row matches a user when every non-null field equals the user's value; a notice is visible if any row matches. The rule exists in Java and in SQL and is checked on every read path, so hiding things in the UI is never the protection.

SHORT

> Visibility is enforced in the backend by one rule: a notice is visible if any target row matches the user.

Follow-ups: What happens if I change the id in the URL? · Why is it written twice?

Q8 What can the Super Admin do that the Principal cannot?

DETAILED

The Super Admin manages users (create, edit, suspend, close), departments, categories and the audit log, sees every notice, and can delete any notice. The Principal is an academic authority: publishes to any audience without approval, approves HODs, and manages class teachers, but does not administer accounts or master data.

SHORT

> The Super Admin administers users, departments, categories and the audit log and sees everything; the Principal publishes and approves.

Follow-ups: Can the Principal see a draft written by an HOD? · Who can read the audit log?

B · Architecture and Technologies

Q9 Explain the architecture of the project.

DETAILED

It is three-tier: a React single-page app, a Spring Boot REST API, and MySQL plus Cloudinary. Inside the backend it is layered: security filter chain → controllers → services → repositories → database. A layer calls only the one beneath it. Controllers contain no business logic; services hold every rule inside transactions; repositories talk to MySQL.

SHORT

> Three tiers, and inside the backend: filter chain, controller, service, repository, database, each calling only the next.

Follow-ups: Where would you add a new rule? · Why can't a controller call a repository?

Q10 Why Spring Boot and React instead of a server-rendered framework?

DETAILED

The same API serves the public site, the portal and, later, a mobile app. React gives reusable components (notice cards, audience builder, file drop) and rich interactions such as the live recipient count. Spring Boot gives mature security, validation, JPA and scheduling.

SHORT

> A separate API lets one backend serve several clients, and React gives reusable, interactive components.

Follow-ups: What are the costs of an SPA? · How does the SPA handle SEO for public notices?

Q11 Why MySQL?

DETAILED

The data is relational and the design relies on foreign keys and unique constraints (one read per notice and user, one class teacher per class, department name unique per wing). MySQL 8 is widely available and a Compose file starts it in one command.

SHORT

> Relational data with constraints; MySQL is common and easy to run with Docker.

Follow-ups: Would the SQL specifications work on PostgreSQL? · What is MySQL-specific in the code?

Q12 Why do you return DTO records instead of entities?

DETAILED

Entities carry internal fields and lazy relations; returning them would leak data (for example the public site must never see creator ids or targets) and would tie the API to the table design. Java 17 records give short, immutable DTOs. `NoticeMapper` builds them, and `PublicDtos` is a separate set so internal fields cannot leak by accident.

SHORT

> To avoid leaking internal fields and to decouple the API from the tables; records make DTOs short and immutable.

Follow-ups: Where are the mappers? · What does the public detail DTO leave out?

Q13 What does each layer of the backend do?

DETAILED

The filter chain authenticates the request and applies URL rules. Controllers map URL and method to a Java method and run `@Valid` and `@PreAuthorize`. Services hold the rules, authorisation checks, transactions, audit and notifications. Repositories are Spring Data interfaces plus Specifications. The database enforces structure.

SHORT

> Filter authenticates, controller maps and validates, service holds the rules, repository stores, database enforces integrity.

Follow-ups: Which layer opens the transaction? · Which layer writes the audit log?

Q14 Why JWT instead of server-side sessions?

DETAILED

JWT keeps the API stateless: no session store, and any instance can serve any request. The browser sends it in an Authorization header. The cost is that a token cannot be revoked before it expires; we reduce that by reloading the account on every request so a suspended user is blocked immediately, and by listing refresh tokens as an improvement.

SHORT

> Stateless and simple to scale; the cost is no revocation, which we soften by checking the account on each request.

Follow-ups: How long does a token last? · What would a refresh token add?

Q15 What is a JPA `Specification` and why did you use it?

DETAILED

A Specification is a reusable, composable query condition built with the Criteria API. The notice list has up to twelve optional filters plus the visibility rule; composing them with `.and()` produces one SQL query with database-side filtering and paging, instead of loading rows and filtering in Java.

SHORT

> A composable query condition; it lets optional filters and the visibility rule combine into one SQL query.

Follow-ups: What does `targetedTo` generate? · How is user input escaped in search?

Q16 What does `spring.jpa.hibernate.ddl-auto=update` do and what is its risk?

DETAILED

It creates missing tables and adds missing columns at start-up from the entities. It never drops anything and never fixes data, so renaming a column or changing a constraint needs manual work; that is why we wrote two upgrade runners. For production, Flyway or Liquibase is the proper approach.

SHORT

> It creates and extends tables automatically; it cannot migrate data, so production should use Flyway.

Follow-ups: What did your runners fix? · What would you do for a column rename?

Q17 Why is pagination done on the server?

DETAILED

A college accumulates thousands of notices. Sending them all would be slow and would expose more than needed. The server takes `page` and `size` (size clamped to 100), the database sorts and limits, and the response carries `totalElements` and `totalPages` so the UI can show page controls.

SHORT

> To keep responses small and fast; the database does the sorting and limiting.

Follow-ups: What if size=100000? · What is the default sort for the feed?

Q18 What does the `CloudStorageService` interface give you?

DETAILED

It hides the Cloudinary SDK behind four methods (upload, delete, signed download URL, is configured). The rest of the code and the tests do not depend on the SDK, so the integration tests use a mock and need no internet, and the storage provider could be replaced.

SHORT

> A seam between our code and Cloudinary, so tests use a mock and the provider could change.

Follow-ups: What happens with no Cloudinary credentials? · How do the tests fake it?

C · Authentication and Security

Q19 Walk me through what happens when I sign in.

DETAILED

The email is normalised and the rate limiter is checked. The user is looked up; if absent, BCrypt still compares against a dummy hash. A wrong password records a failure and returns a generic 401. A correct password clears the counter; then the account status is checked. If it is not APPROVED we return 403 with an `account` object and no token; otherwise `JwtService` issues a signed token and we return it with the user.

SHORT

> Rate-limit check, BCrypt compare, generic 401 on failure, status check only after a correct password, then a signed JWT.

Follow-ups: Why check the status after the password? · What is in the 403 body?

Q20 Why return the same error for an unknown email and a wrong password?

DETAILED

If the messages differed, an attacker could test thousands of emails and learn which have accounts. The code returns the identical 401 and does the same BCrypt work either way, so the response time is similar. The forgot-password endpoint likewise always says "If the email is registered, a reset link has been sent".

SHORT

> To stop attackers discovering which emails are registered.

Follow-ups: What is the dummy hash for? · Does the registration form leak existence?

Q21 Why does a pending account get a 403 rather than a 401?

DETAILED

The credentials are correct, so authentication succeeded; what fails is permission to proceed. 403 with an `account` object lets the sign-in page explain what happens next (who it is waiting for, the rejection or suspension reason, a timeline) without issuing a token.

SHORT

> The login was correct but the account may not proceed, so 403 with an explanation.

Follow-ups: What does a rejected applicant see? · Can they apply again?

Q22 What is inside the JWT? Can a user change their role by editing it?

DETAILED

The subject is the email, with `uid`, `role`, issue time and expiry. Anyone can read the payload but cannot change it: it is signed with HMAC-SHA-512 and a server-only secret, so an edited token fails verification. Moreover, the server does not trust the `role` claim; it reloads the user from the database and uses that.

SHORT

> Email, user id, role, expiry, signed with HS512. Editing breaks the signature, and the server uses the database role anyway.

Follow-ups: Why is the secret at least 64 bytes? · What if two roles differ in DB and token?

Q23 Why do you load the user from the database on every request?

DETAILED

So that suspension, closure and role or department changes take effect immediately. If the server trusted the token, a user suspended an hour ago would keep working until the token expired. The cost is one lookup by email per request. A test proves a suspended user loses access at once.

SHORT

> So blocking or changing an account is instant, even with a valid token.

Follow-ups: Isn't that slow? · How does this relate to stateless?

Q24 How are passwords stored and what is the password policy?

DETAILED

Passwords are hashed with BCrypt, which is salted and deliberately slow; the plain password is never stored or logged. The policy is 8–64 characters with an uppercase letter, a lowercase letter, a digit and a symbol, enforced by one regex on the server and mirrored by the browser's checklist.

SHORT

> BCrypt hashes; 8–64 characters with upper, lower, digit and symbol, enforced by the server.

Follow-ups: Why a maximum length of 64? · What is a salt?

Q25 How does password reset work and why do you store only a hash of the token?

DETAILED

A random 32-byte token is emailed in a link; the database stores its SHA-256 and a 30-minute expiry. On reset the submitted token is hashed and looked up. If the database leaked, the hashes cannot be turned into working links. After use the token is cleared so it cannot be replayed.

SHORT

> A random single-use token, stored hashed, expiring in 30 minutes.

Follow-ups: Why SHA-256 and not BCrypt for the token? · What if mail is disabled?

Q26 How does rate limiting work?

DETAILED

`AuthRateLimiter` keeps sliding windows in memory: 5 failed sign-ins per email and address, 20 per address, in 15 minutes, and 10 registrations per address per hour. A success clears the email counter. Counting per email **and** address means a stranger cannot lock the real owner out from their own network.

SHORT

> In-memory sliding windows per email+address and per address; 5 failures or 20 per address in 15 minutes.

Follow-ups: What is the weakness behind a proxy? · How would you scale it?

Q27 Why have you disabled CSRF protection?

DETAILED

CSRF relies on credentials the browser attaches automatically, such as cookies. This API uses none: the token is added to an Authorization header by JavaScript, which a cross-site form cannot do. If we moved the token into a cookie we would have to turn CSRF protection back on.

SHORT

> No cookies are used, so there is nothing for a cross-site request to ride on.

Follow-ups: What is the risk of localStorage? · What would change with cookies?

Q28 What happens if a token is stolen?

DETAILED

It works until it expires (24 hours) or the account is suspended or closed, because logout is stateless and only deletes the browser's copy. This is a known limitation. The standard fix is short-lived access tokens with a revocable refresh token, at the cost of server-side state.

SHORT

> It stays valid until expiry or until the account is blocked; short tokens plus refresh tokens would fix it.

Follow-ups: Can an admin kill a session? · How would you implement refresh tokens?

Q29 What is CORS and how is it configured?

DETAILED

Browsers block cross-origin calls unless the server allows the origin. `SecurityConfig` allows only the origins listed in `app.cors.allowed-origins`, the methods GET, POST, PUT, PATCH, DELETE and OPTIONS, and the Authorization and Content-Type headers, with a one-hour preflight cache. CORS protects browsers; it does not replace server-side authorisation.

SHORT

> Only configured origins, methods and headers are allowed. It protects browsers, not the API itself.

Follow-ups: Does CORS stop curl? · What are the production origins?

Q30 A student uses curl to call `POST /api/notices`. What stops them?

DETAILED

Three things. The JWT filter authenticates them as a student. `@PreAuthorize` on the endpoint allows only Super Admin, Principal, HOD and Faculty, so Spring answers 403. Even if that were missed, `NoticeService.create` and `TargetPolicy` throw "Students cannot create notices". A test covers it.

SHORT

> The endpoint's role rule gives 403, and the service independently refuses students.

Follow-ups: Why check twice? · Which test covers it?

Q31 What are the roles of `@PreAuthorize`, `TargetPolicy` and `NoticeAccessService`? Why three?

DETAILED

They answer different questions. `@PreAuthorize`: may this **role** call this endpoint? `TargetPolicy`: may this person address **this audience**? `NoticeAccessService`: may this person act on **this specific notice** (view, edit, publish, review)? The first needs only the role; the second needs the request; the third needs the stored notice. Combining them would blur the questions.

SHORT

> Endpoint by role, audience by role and department, and action by relationship to the notice.

Follow-ups: Give an example that passes the first but fails the third.

Q32 Why does a private notice you may not see give 403, but a non-public notice on the public API gives 404?

DETAILED

For members, 403 says "this exists but is not for you", which is acceptable once signed in. For anonymous visitors the existence of an unpublished or private notice is itself information, so the public API answers 404 exactly as for an id that does not exist.

SHORT

> Members get 403; the anonymous public API gets 404 so it never confirms that a hidden notice exists.

Follow-ups: Would 404 also be safe for members? · What does the public API leak otherwise?

D · Targeting and Visibility

Q33 Explain `NoticeTarget` and the matching rule.

DETAILED

Each target row has nullable role, department, semester, class and user. A row matches a user when every non-null field equals the user's value, and a notice is visible if any row matches. Null means "any". So `COLLEGE` is all nulls (everyone), `ROLE HOD` is only the role set, and `CLASS` has role, department, semester and class set.

SHORT

> A row matches when all its non-null fields equal the user's; a notice is visible if any row matches.

Follow-ups: How would you target 'CS semester 6 or Mechanical semester 4'? · What does an empty row mean?

Q34 Why store the audience as rows instead of a role list on the notice?

DETAILED

Audiences are combinations: "HODs and faculty", "CS 6A and IT 6A". Rows with nullable columns express any combination with one matching rule and need no schema change for a new shape, and the same rows give readable labels ("Computer Engineering semester 6 class A").

SHORT

> Rows express any combination with one rule, and need no schema change for new audience shapes.

Follow-ups: What is the cost? · How are duplicates handled?

Q35 How does the notice list apply visibility inside SQL?

DETAILED

`NoticeSpecifications.targetedTo(user)` builds an EXISTS sub-query on `notice_targets` for the notice, with conditions "target role is null or equals mine", "department is null or equals mine", and so on for semester, class and user. The database filters and pages, and only visible notices are ever loaded.

SHORT

> An EXISTS sub-query on the targets table that matches my role, department, semester, class and id.

Follow-ups: What if the user has no department? · What index helps?

Q36 Why is the matching rule implemented twice?

DETAILED

Java (`TargetMatcher`) answers questions about one notice and one user, such as `canView` or `isRecipient`. SQL answers questions about all notices, such as the feed and dashboard counts. Loading every notice to filter in Java would not scale. We keep the rule tiny and test both against the same cases.

SHORT

> Java for one-to-one checks, SQL for list queries; the rule is small and tested in both.

Follow-ups: How do you stop them drifting apart? · Which tests?

Q37 A student changes semester after a notice was published. What do they see?

DETAILED

Visibility is evaluated at **read time** against the user's current values, so they immediately stop seeing semester-6 notices and start seeing semester-7 ones. Notifications created at publish time are not recomputed. That is a deliberate trade-off.

SHORT

> Visibility is computed when they read, using current values; old notifications are not recalculated.

Follow-ups: Is that desirable? · How would a promotion process affect it?

Q38 How does the live recipient count avoid leaking information?

DETAILED

`POST /notices/recipient-preview` runs the targets through `TargetPolicy.buildTargets` exactly as saving does, so an HOD asking for another department's size gets the same 403 as saving would. A test checks that the preview applies the same rules as saving.

SHORT

> It validates and authorises the audience exactly like saving, so it cannot reveal forbidden audiences.

Follow-ups: Why debounce the call? · What does the count exclude?

Q39 Who can address which audience?

DETAILED

Super Admin and Principal: anyone. HOD: only their own department (department, role in department, semester, class, specific users), never the whole college or a bare role. Faculty: only students of their own department by department, semester, class or specific students. Students: none. The UI offers only these, and the server enforces them.

SHORT

> Admin and Principal anyone; HOD their department; Faculty their department's students; students nobody.

Follow-ups: Can an HOD notify other HODs? · How are 'specific users' restricted?

Q40 What does an empty list of targets mean?

DETAILED

Nobody. A private notice with no target is rejected with "Select at least one target audience". A public notice may have none, and `RecipientService` returns an empty list at once, because an OR over zero conditions could otherwise degrade into "everyone".

SHORT

> Nobody; empty is never treated as everyone.

Follow-ups: Which test covers this? · Why is the early return important?

Q41 How does 'specific users' targeting work without exposing the user list?

DETAILED

`GET /users/lookup` is scoped by the caller: an HOD sees only their department, a Faculty member only students of their department, students are refused, and results are limited to 20 active users. The final targets are checked again by `TargetPolicy` when saved.

SHORT

> A scoped, limited lookup, with the final choice re-checked on save.

Follow-ups: Why 20 results? · Why only active users?

Q42 What about deactivated users when resolving recipients?

DETAILED

`RecipientService` filters by the `active` flag, so only approved accounts are counted or notified, and the creator is excluded. A deactivated user cannot sign in anyway. Selecting an inactive user as a specific target is rejected.

SHORT

> Only active users are recipients; the creator is excluded.

Follow-ups: What is `active` versus `status`? · What if an account is suspended later?

Q43 How are read and acknowledgement statistics computed?

DETAILED

`statsFor` resolves the recipient ids for the notice, then intersects them with the ids that have a read row and those with an acknowledgement row. Total, read, not read, acknowledged and not acknowledged are all relative to intended recipients, so a stray reader cannot inflate them.

SHORT

> Read and acknowledged ids are intersected with the intended recipients.

Follow-ups: Why store reads separately from acknowledgements? · What is the scaling issue?

Q44 Can the creator and others besides the audience see a notice?

DETAILED

Yes. `canView` is true for the Super Admin, the creator, and (for faculty notices that are no longer drafts) the reviewing HOD, plus recipients when the notice is published or expired. Public notices are also visible to anyone while live. The creator is excluded from notifications and from read statistics.

SHORT

> The Super Admin, the creator and the reviewing HOD, in addition to the recipients.

Follow-ups: Why can the HOD see a faculty draft? (they cannot, only non-drafts) · Can the Principal see an HOD's draft?

E · Approval and Lifecycle

Q45 Describe the notice statuses and transitions.

DETAILED

DRAFT is the start for every notice, including AI drafts. Principal, HOD and Admin publish directly: DRAFT → PUBLISHED, or → SCHEDULED if the publish date is in the future and the scheduler publishes it later. Faculty submit: DRAFT → PENDING_APPROVAL → (HOD approves) APPROVED → PUBLISHED or SCHEDULED; or REJECTED, then editing returns it to DRAFT. PUBLISHED → EXPIRED by the scheduler; PUBLISHED or EXPIRED → ARCHIVED by the creator or Admin.

SHORT

> Draft; publish or schedule; faculty go through pending approval; published then expired then archived.

Follow-ups: Is APPROVED ever visible? · Can a published notice be edited?

Q46 Why do faculty notices need approval but HOD notices do not?

DETAILED

Faculty notices reach students under the department's name, so the HOD acts as quality control. HODs are themselves the department authority and are limited to their own department by `TargetPolicy`; the Principal and Admin have authority over the whole college. The system still records every publish in the audit log.

SHORT

> Faculty are reviewed by their HOD; HODs, the Principal and Admin are trusted to publish, within their scope.

Follow-ups: Could you make HOD notices need Principal approval? · Where would you change it?

Q47 What happens when an HOD rejects a notice?

DETAILED

The reason is mandatory (up to 1,000 characters). The notice becomes REJECTED, stores the reason, reviewer and time, and the author is notified with the reason. Editing it turns it back into a DRAFT, which can be resubmitted to PENDING_APPROVAL. The timeline shows the reason.

SHORT

> It needs a reason, becomes REJECTED, the author is told, and can edit and resubmit.

Follow-ups: Who sees the reason? · Is the old reason cleared on resubmit?

Q48 How does scheduling and expiry work?

DETAILED

A Spring `@Scheduled` job runs one minute after the previous run completes. It publishes SCHEDULED notices whose publish date has arrived (or expires them if their expiry already passed) and expires PUBLISHED notices past their expiry. Feeds also filter by the dates, so a notice stops being shown the moment it expires even if the job has not run yet.

SHORT

> A job every minute publishes due notices and expires old ones; queries also filter by date.

Follow-ups: Why `fixedDelay` rather than `fixedRate`? · What if the server was down?

Q49 What if the server was down when a notice should have been published?

DETAILED

The next run uses `publishDate <= now`, so it publishes everything that became due while it was down. If the expiry also passed during the outage, the notice goes straight to EXPIRED with the audit note "Expired before it was published". Recipients are never notified about a notice that is already expired.

SHORT

> It catches up on the next run; one that already expired is marked expired, not published.

Follow-ups: How does the feed behave before the job runs? · Is the audit entry written by a user?

Q50 Why is APPROVED a transient status?

DETAILED

On approval the service records the reviewer, sets APPROVED, and in the same transaction calls the same release-or-schedule logic as publish, which sets PUBLISHED or SCHEDULED. So APPROVED is written but never rests. The audit log still contains the NOTICE_APPROVED event, which feeds the timeline.

SHORT

> Approval immediately publishes or schedules in the same transaction, so APPROVED never rests.

Follow-ups: Why keep the status at all? · What does the timeline show?

Q51 Can a faculty member edit a notice after submitting it?

DETAILED

No. `canEdit` allows faculty to edit only DRAFT or REJECTED notices. After submission the notice is locked while it waits for the HOD, so the reviewer sees exactly what will be published. Other creators can edit drafts, scheduled and published notices.

SHORT

> No; faculty can edit only drafts and rejected notices, so the reviewer sees what will be published.

Follow-ups: Can an HOD edit a published notice? · Are recipients told of the change?

Q52 Who can delete a notice and what is removed?

DETAILED

The Super Admin can delete any notice; a creator can delete their own (faculty only while DRAFT or REJECTED). The service deletes Cloudinary files first, then notifications, acknowledgements, reads and saved entries, then the notice with its targets, and writes a NOTICE_DELETED audit entry.

SHORT

> Admin any, creators their own; the cloud files go first and then all dependants.

Follow-ups: What if Cloudinary is down? · Is the audit entry removed?

Q53 Where does the approval timeline come from?

DETAILED

`NoticeTimelineService` reads the audit log entries for the notice (created, submitted, approved, rejected, scheduled, published, expired, archived), maps actors to names and roles, merges consecutive identical edits into 'Edited, 3 times', and adds a 'next step' line in the UI. Only the creator, reviewing HOD and Super Admin may see it, so recipients do not see internal review history.

SHORT

> It is built from the audit log, so there is one source of truth for who did what.

Follow-ups: Why not a separate history table? · What does a recipient see?

Q54 What does archive do?

DETAILED

It sets ARCHIVED for a PUBLISHED or EXPIRED notice; only the creator or the Super Admin may do it. Recipients then stop seeing it in their feed and history, which only include PUBLISHED and EXPIRED, but the notice and its audit trail remain.

SHORT

> It hides a finished notice from readers while keeping it and its history.

Follow-ups: Can an archived notice be restored? · Is it deleted from saved lists?

F · Database and JPA

Q55 Describe the database design.

DETAILED

Twelve tables. Core: users, departments, notices, notice_categories. Audience and files: notice_targets, attachments. People and workflow: class_teachers, notifications. Engagement: notice_reads, notice_acknowledgements, saved_notices. Accountability: audit_logs. A department has many users; a user creates many notices; a notice has one category and many targets, attachments, notifications, reads, acknowledgements and saves.

SHORT

> Twelve tables: users, departments, notices, categories, targets, attachments, class teachers, notifications, reads, acknowledgements, saves, audit log.

Follow-ups: Which tables have unique constraints? · Draw the ER diagram.

Q56 Why unique constraints on reads, acknowledgements and saves?

DETAILED

`(notice_id, user_id)` is unique in each, so a person cannot be recorded twice even if two requests race. The database is the final guard, and the service additionally checks existence first for a friendly path.

SHORT

> So each person has at most one read, acknowledgement or save per notice, even under concurrent requests.

Follow-ups: What happens on a violation? · Why check in code as well?

Q57 Why does `audit_logs` have no foreign keys?

DETAILED

History must outlive the things it describes. The actor is stored as a plain id plus the email at the time, and the entity as a type and id. Deleting a notice or deactivating a user never breaks or erases the log, and the system itself can be the actor (null id).

SHORT

> So the log survives deletes and can record system actions.

Follow-ups: How do you show names for old entries? · Can the log be edited?

Q58 Why are users deactivated instead of deleted?

DETAILED

Notices, audit entries and statistics reference users. A hard delete would orphan them or cascade away history. Deactivation (status DEACTIVATED) blocks sign-in and removes the person from recipient lists while preserving everything else. It can be reversed.

SHORT

> To keep notices, audit history and statistics consistent; closing is reversible.

Follow-ups: What does DELETE /users/{id} actually do? · Can a user be reactivated?

Q59 Why are enums stored as strings?

DETAILED

`@Enumerated(EnumType.STRING)` stores the name, so re-ordering or inserting an enum value can never change the meaning of existing rows, which ordinal storage would. It also makes the database readable.

SHORT

> So changing the enum order cannot corrupt existing data.

Follow-ups: What is the cost? · How long are the columns?

Q60 What are `active` and `status` on the user, and why both?

DETAILED

`status` is the full lifecycle (PENDING, APPROVED, REJECTED, SUSPENDED, DEACTIVATED). `active` is the older boolean that many queries and Spring Security use. The entity keeps them in step so that `active == (status == APPROVED)`, and old rows without a status are read from `active`.

SHORT

> Status is the lifecycle; active is kept equal to 'is approved' so older queries still work.

Follow-ups: What does the backfill runner do? · Where is the sync done?

Q61 What is the N+1 problem and where did you avoid it?

DETAILED

Loading a page of 12 notices and then running separate queries per notice for attachment count, read flag and saved flag causes 1 + 36 queries. `NoticeService.summaries` fetches those three things for the whole page in three batched queries and joins them in memory.

SHORT

> One query per item multiplies; we batch the counts for a whole page instead.

Follow-ups: What fetch type do the relations use? · How would you detect N+1?

Q62 Which indexes exist and why?

DETAILED

`users(role)` and `users(department_id, semester, class_name)` for recipient lookups; `notices(status)` for the scheduler and lists; `notice_targets(notice_id)` for the visibility sub-query; `notifications(user_id)` and `saved_notices(user_id)` for per-user lists; `audit_logs(event_time)` for newest-first browsing; plus the unique indexes.

SHORT

> On the columns the visibility, scheduler and per-user queries filter by.

Follow-ups: Is the text search indexed? · What would you add at scale?

Q63 How is the department name unique per wing but the code unique overall?

DETAILED

The table has a unique constraint on `(name, institution_type)` and a unique `code`. So Engineering and Polytechnic can both have "Computer Engineering" with different codes (CS and DCO). Older databases had a unique index on name alone, which `InstitutionUpgrade` drops on MySQL at start-up.

SHORT

> A composite unique on (name, wing) and a global unique code.

Follow-ups: Why did the old index need dropping? · How did you find its name?

Q64 Why is the notice description a TEXT column?

DETAILED

Notices can be long (up to 20,000 characters by validation), more than a `VARCHAR(255)` default holds. TEXT stores it without a fixed small limit; the title stays a 200-character VARCHAR because it is shown in lists and searched often.

SHORT

> Bodies are long, so TEXT; titles are short and stay VARCHAR(200).

Follow-ups: Is TEXT searchable? · What are the limits?

G · Files, Notifications and the AI Assistant

Q65 How are attachments protected?

DETAILED

Files are validated by extension, MIME type, size and magic bytes, stored in Cloudinary with delivery type `authenticated` (so the stored URL does not work alone) under a random UUID name, and downloaded only through a 5-minute signed link issued after `canView` passes. The URL is never returned by the API.

SHORT

> Strict validation, private storage, and 5-minute signed links after a visibility check.

Follow-ups: What if someone shares a link? · Why not make files public?

Q66 Why check the file's first bytes (magic bytes)?

DETAILED

A file's extension and declared content type are chosen by the sender. Renaming `virus.exe` to `.pdf` passes both. Checking that the first bytes are `%PDF`, the PNG header, FF D8 FF, the OLE2 header or the ZIP header proves the content looks like the claimed type. A unit test rejects a renamed executable.

SHORT

> Extension and MIME type can be faked; the signature cannot be faked cheaply.

Follow-ups: Why is a .docx a ZIP signature? · Is that enough against malware?

Q67 Why delete from Cloudinary before deleting the database row?

DETAILED

Two systems cannot share a transaction. If we deleted the row first and Cloudinary failed, we would have an untracked file forever. Cloud first means a failure leaves the row and the user gets 502 and can retry; "not found" counts as success, so retries are safe.

SHORT

> So a failure leaves a retryable state instead of an orphaned file.

Follow-ups: What if it fails after the cloud delete? · What about upload failures?

Q68 How does a notice reach thousands of people?

DETAILED

At publish, `RecipientService` resolves the active users matching any target row with one Specification query, excluding the creator. One Notification row is saved for each (batch `saveAll`). If the priority is Important or Urgent, `EmailService` sends one email per recipient on a background pool. The audit entry records the number of recipients.

SHORT

> One query resolves recipients, one notification row each, and emails run in the background.

Follow-ups: What is the scaling limit? · How would you use a queue?

Q69 Why are emails sent only for Important and Urgent notices?

DETAILED

Email is intrusive and mailboxes fill up. Normal notices appear in the app and notifications; email is reserved for what must be seen. It also keeps the load on the SMTP account small. Mail is optional and disabled by default.

SHORT

> To keep email meaningful and avoid spamming people with routine notices.

Follow-ups: Can a recipient opt out? · What does Gmail require?

Q70 Why is mail sent asynchronously?

DETAILED

Sending hundreds of emails inside the publish request would make it slow and a mail outage would fail the publish. `@Async("mailExecutor")` runs sending on a pool of 2–4 threads with a queue of 5,000; each recipient's failure is caught and logged separately.

SHORT

> So publishing returns quickly and a mail failure cannot break it.

Follow-ups: What if the queue fills? · Is delivery guaranteed?

Q71 How does the AI assistant work, and what if there is no API key?

DETAILED

The creator types a sentence. If `AI_API_KEY` is set, we call the Anthropic Messages API with a system prompt that lists the active categories and asks for only JSON; otherwise, or on any failure, an offline keyword-based drafter produces a similar suggestion. The result is sanitised and saved as a draft; the response says whether the source was AI or MOCK.

SHORT

> An LLM call that returns JSON, with an offline rule-based fallback, always saved as a draft.

Follow-ups: What does 'sanitise' do? · How is the audience chosen?

Q72 Why is AI output never published automatically?

DETAILED

A model can be wrong, and notices carry the college's authority. The draft goes through the same publish or approval workflow as any other notice, with the creator reviewing it. The audience suggestion is used only if the creator's role is allowed to use it. A test asserts AI notices are drafts.

SHORT

> Because a human must review it; it enters the normal workflow as a draft.

Follow-ups: What if the model suggests a forbidden audience? · How is it marked in the UI?

Q73 What if the AI returns invalid JSON or a strange date?

DETAILED

The client strips code fences, parses enums leniently with fallbacks, and parses dates tolerantly. Any exception returns empty and the mock drafter is used. `sanitise` then limits the title length, falls back to the user's text for an empty body, discards past publish dates, and defaults the expiry to seven days.

SHORT

> It falls back to the offline drafter, and sanitise forces safe values.

Follow-ups: Does the API key ever reach the browser? · What are the timeouts?

H · Frontend

Q74 How does the UI know which buttons to show on a notice?

DETAILED

The detail response includes a `permissions` object computed by `NoticeAccessService.permissions` (canEdit, canPublish, canSubmit, canReview, canAcknowledge, canViewStats, canArchive, canDelete, canSave). The page renders exactly those buttons, so the UI never has its own copy of the rules.

SHORT

> The server returns a permissions object and the page shows only what it allows.

Follow-ups: What if the UI shows a button wrongly? · Does hiding a button secure anything?

Q75 If the server checks everything, why have `ProtectedRoute`?

DETAILED

For user experience: an anonymous visitor is sent to the login page and remembered, and a student does not land on an admin page that would only show errors. It is not security; if someone bypasses it, every API call is still rejected.

SHORT

> It is a convenience for the user; the server enforces access.

Follow-ups: What happens if I edit the JS in dev tools? · Where is the token kept?

Q76 How can `/` be both a public landing page and the dashboard?

DETAILED

`RootFrame` and `Home` choose by the signed-in state: members get the portal `Layout` with the dashboard, visitors get `PublicLayout` with the landing page. Public routes such as `/public-notices` are available in both frames, so a member can still open them.

SHORT

> The root route decides by whether a user is signed in.

Follow-ups: What is shown while the session is loading? · How does the page title change?

Q77 What does `useAsync` do?

DETAILED

It takes a loader function and dependencies and returns `{ data, loading, error, reload }`, re-running when the dependencies change. It standardises loading and error states, so pages show skeletons and an error with a *Try again* button without repeating code.

SHORT

> A small hook for loading data with loading, error and reload states.

Follow-ups: Why not Redux or React Query? · How is caching handled?

Q78 How does the draft auto-save work and is it safe?

DETAILED

After 1.5 seconds without typing, the form writes its values to `localStorage` under `cnb_draft:<user>:<notice>`. When the author returns, a banner offers Restore or Discard. Nothing reaches the server until they save, so half-written notices never appear in the audit log. Files cannot be stored, so the banner says to choose them again. The key includes the user id.

SHORT

> A debounced copy in the browser, per user and notice, restorable on return.

Follow-ups: Is localStorage safe on a shared computer? · When is it cleared?

Q79 How does the live audience summary avoid showing stale numbers?

DETAILED

It waits 400 ms after the last change, sends the targets, and tags each request with a sequence number; only the response whose number matches the latest request is displayed, so a slow earlier response can never overwrite a newer one.

SHORT

> Debounce plus a request sequence counter so only the latest answer is shown.

Follow-ups: What does it show on error? · What does it count?

Q80 How is the dark theme applied without a flash?

DETAILED

A tiny inline script in `index.html` reads the saved preference (or the system setting) and sets `data-bs-theme` before React renders. `ThemeContext` then keeps it in sync, including when the operating system switches. The preference is stored per device.

SHORT

> An inline script sets the theme before the first paint.

Follow-ups: What does 'match my device' do? · Where is it stored?

Q81 What did you do for accessibility?

DETAILED

A skip-to-content link, labelled fields with `aria-invalid` and `aria-describedby`, `aria-live` regions for toasts, statuses and the audience summary, visible focus outlines, priority and status shown in text as well as colour, and a mobile layout with a bottom navigation bar and a slide-in drawer.

SHORT

> Skip link, labelled fields, live regions, text as well as colour, and a responsive layout.

Follow-ups: How would you test it? · What is missing?

I · Testing, Deployment and Reflection

Q82 How did you test the project?

DETAILED

72 automated tests: JUnit unit tests for `TargetMatcher`, `FileValidator` and `JwtService`; Mockito tests for `TargetPolicy`; and Spring Boot integration tests with MockMvc on H2 for visibility, approval, public-site leakage, registration routing, rate limiting, attachments and the AI draft. Cloudinary is mocked, so no network is needed.

SHORT

> 72 tests: unit, Mockito and integration tests on an in-memory database.

Follow-ups: Which test is most important? · What is not tested?

Q83 Why use H2 in tests and what is its limit?

DETAILED

H2 in MySQL mode makes the suite fast and self-contained. The limit is that it is not MySQL: for example the start-up step that drops an old MySQL index only runs on MySQL and is not covered. A Testcontainers MySQL would close that gap.

SHORT

> It is fast and self-contained but not identical to MySQL.

Follow-ups: What does Testcontainers add? · What else differs?

Q84 How do I run the project?

DETAILED

Needs Java 17, Node 18 and MySQL 8. Start MySQL (`docker compose up -d` creates the database user from `docker-compose.yml`), run `./mvnw spring-boot:run` in `backend` (port 8080), then in `frontend` copy `.env.example` to `.env`, run `npm install` and `npm run dev` (port 5173). Sign in with a seeded account such as `admin@college.edu` and `Password@123`.

SHORT

> Start MySQL, run the Spring Boot backend and the Vite frontend, then sign in with a seeded account.

Follow-ups: What if Cloudinary is not set up? · What if port 3306 is busy?

Q85 What changes in the production profile?

DETAILED

`application-prod.properties` reads the database, JWT secret, frontend URL and CORS origins from environment variables with no defaults, disables seeding, hides Swagger, and lowers logging to WARN. The Dockerfile builds the JAR in a Maven stage and runs it on a JRE 17 image with the `prod` profile.

SHORT

> Secrets from environment variables, no seed data, no Swagger, quieter logs.

Follow-ups: Why must JWT_SECRET be 64 characters? · What would you add for HTTPS?

Q86 Where is the weakest part of the project, honestly?

DETAILED

Token revocation (logout is client-side), the in-memory rate limiter that uses the direct remote address, the admin path that bypasses the one-HOD rule, no re-notification when a published notice is edited, and statistics that load the recipient set into memory. All are documented in §19 with their fixes.

SHORT

> No token revocation, an in-memory rate limiter, and a few rules not applied on the admin path; all documented.

Follow-ups: Which would you fix first? · How would you prove the fix works?

Q87 If you had another month, what would you build first?

DETAILED

Refresh tokens with revocation, because it closes the largest security gap; then Flyway migrations and a Testcontainers MySQL suite; then push notifications. Those make the existing system safer before adding features.

SHORT

> Refresh tokens and revocation, then proper migrations and real-database tests.

Follow-ups: Why not new features first? · What would the refresh token table look like?

§22 Difficult Faculty Questions

These are the questions a sharp examiner asks to find out whether you understand your own system or merely assembled it. Each is answerable from this project's real implementation, and several are best answered by **admitting a limitation**, which is far stronger than bluffing.

D1 "Your React app hides the Publish button from students. Why is that not enough?"

IDEALANSWER

Because the browser is under the user's control. Anyone can open developer tools, edit the page or skip the browser entirely and call `POST /api/notices/5/publish` with `curl`. So the real protection is on the server, in three layers: the endpoint's `@PreAuthorize` role rule, `TargetPolicy` for the audience, and `NoticeAccessService` for the specific notice. The UI only shows what the server's `permissions` object says the caller may do, so it is a courtesy, never the control.

> This is the highest-value answer in the viva. Say "the UI mirrors; the server decides" and offer to demo a curl call that gets 403.

D2 "You wrote the visibility rule in Java and in SQL. Isn't duplicated logic a bug waiting to happen?"

IDEALANSWER

Yes, it is a real risk, and I would not claim otherwise. We accepted it for a reason: list pages must be filtered inside the database for performance, while single-notice checks are naturally in Java. We reduced the risk by keeping the rule to one sentence (every non-null field equals the user's; any row matches), by cross-referencing the two implementations in their comments, and by testing that lists, details and recipient counts agree. The alternative, which I would consider if the rule grew, is to run single checks through the same SQL Specification (a count query for one id), leaving a single implementation.

D3 "A JWT payload can be read by anyone. How can it be secure?"

IDEALANSWER

A JWT gives **integrity**, not confidentiality, and that is the right property here. The payload holds an email, a user id and a role, nothing secret. What matters is that nobody can change it: the signature is an HMAC-SHA-512 with a secret only the server holds, so editing `role` breaks it. And we do not even trust the role claim: every request reloads the user from the database and uses the stored role.

D4 "What if someone steals a token? Can you revoke it?"

IDEALANSWER

Not directly, and that is the honest cost of stateless tokens. Logout only deletes the browser's copy, so a stolen token works until it expires (24 hours). Two things reduce the damage: because we reload the account on every request, an administrator can **suspend or close the account** and the token stops working immediately; and the secret is long and validated at start-up. The proper fix is a short-lived access token plus a refresh token stored server-side, which gives real revocation at the price of some state.

> Admitting this scores far better than claiming the design is flawless.

D5 "You disabled CSRF protection. Isn't that a vulnerability?"

IDEALANSWER

CSRF exploits credentials the browser attaches **automatically**, which means cookies. Our token is kept in `localStorage` and attached by our own JavaScript in an `Authorization` header, which a cross-site form cannot set. With no automatic credential there is no CSRF vector. The trade-off is that `localStorage` is readable by injected script (XSS), so React's escaping matters. If we moved to an `httpOnly` cookie, we would have to enable CSRF protection again.

D6 "Your rate limiter is a map in memory. What breaks with two servers, or behind a reverse proxy?"

IDEALANSWER

Two things. With two instances each keeps its own counters, so an attacker effectively gets twice the attempts, and a restart clears them. Behind a proxy, `getRemoteAddr()` returns the proxy's address, so every user shares one counter and 20 failures from anyone could lock everyone out. The class comment says so. The fix is a shared store such as Redis and reading the client address from a trusted `X-Forwarded-For` header that the proxy sets.

> Knowing the limit of your own component, and where it is written down, is what the examiner wants to hear.

D7 "If you load the user from the database on every request, why use JWT at all?"

IDEALANSWER

JWT still does useful work: it proves the caller authenticated recently without a session store, carries the id so the lookup is a primary-key or unique-index read, and keeps the API stateless in the sense that no server memory holds a session. We deliberately trade one cheap read per request for **immediate** effect of suspension and role changes. If load demanded it, we could cache the account for a few seconds and accept that delay.

D8 "Can the administrator create two HODs for the same department?"

IDEALANSWER

Yes, today they can, and that is a gap I know about. Self-registration enforces "one approved HOD per department" through `ensureHodVacancy`, but `UserService.create` and `update` do not call it. The fix is one line in `applyRoleData` for the HOD case. I would add a test first that fails, then the fix.

> A clearly-described gap with a precise fix is better than hoping the examiner does not find it.

D9 "An HOD edits a published notice. Are recipients told?"

IDEALANSWER

No. `canEdit` allows non-faculty creators to edit PUBLISHED notices, and the edit is audited (`NOTICE_EDITED`) and shown on the timeline, but recipients are not re-notified, and a changed audience does not notify newly included people. For a notice board I think that is a real weakness: a changed exam date should alert people. The improvement would be to compare old and new and notify, or to treat material changes as a new version.

D10 "Cloudinary and MySQL are two systems. How do you keep them consistent?"

IDEALANSWER

We cannot have one transaction across both, so we order operations so that the remaining failure is harmless. Upload: validate everything first, upload, and if anything fails delete what was already uploaded. Delete: remove the cloud file **first**; if that fails, keep the row and return 502 so the user can retry; and treat "not found" as success so retries are idempotent. The worst remaining case is a crash between a successful cloud upload and the database commit, which could leave one orphan; a periodic reconciliation job would clean that up.

D11 "Does 'I have read this notice' prove the person read it?"

IDEALANSWER

No. It proves that they pressed a button after the notice was shown to them, and records when. That is useful accountability (we know who has not even confirmed) but it is not proof of understanding. We are careful in the wording of statistics: 'read' means opened, 'acknowledged' means confirmed. Stronger evidence would need a quiz or a signature step, which is outside this project's scope.

D12 "You use `ddl-auto=update`. Would you deploy that to production?"

IDEALANSWER

No. It is convenient in development and it is why the project starts with an empty database, but it cannot rename columns, migrate data or drop constraints, and a wrong entity change could alter a live schema silently. We needed to hand-write two start-up runners to upgrade older data, which is exactly the job migration tools do properly. For production I would use Flyway with versioned SQL scripts and set `ddl-auto=validate`.

§23 Complete Presentation Script

Word-for-word script for all 16 slides, with speaker assignments. Total speaking time about 15 minutes, plus 6–8 minutes for the live demonstration. **Do not read this aloud verbatim on the day**: read it enough times that the ideas become yours, then speak naturally.

Speaker	Slides	Approximate time
Speaker 1	1 Title · 2 Index · 3 Problem Statement · 16 Conclusion	3 min 30 s
Speaker 2	4 Objectives · 5 Existing System · 6 Proposed System · 15 Future Scope	3 min 30 s
Speaker 3	7 Technologies · 8 System Architecture · 9 Database Design	3 min 30 s
Speaker 4	10 Major Modules I · 11 Major Modules II · 12 Security and Reliability	3 min
Speaker 5	13 Live Demonstration · 14 Challenges Faced	8 min including the demo

Slide 1 — Title · Speaker 1 · 30 seconds

"Good morning, respected faculty members. We are presenting the College Online Notice Board System. We are a team of five, guided by our project guide. It is a full-stack web application built with Spring Boot, React and MySQL, and over the next fifteen minutes we will show the problem it solves, how we designed it, and then run it live."

Slide 2 — Index · Speaker 1 · 20 seconds

"We begin with the problem and our objectives, compare the existing notice board with our proposed system, then move into the technical side: technologies, architecture and database. After that we cover the major modules and security, one of us will demonstrate it live, and we close with the challenges we faced and future scope."

Slide 3 — Problem Statement · Speaker 1 · 1 minute 20 seconds

"A physical notice board is one sheet of paper on one wall, and that creates problems for everyone.

For students: a notice is only seen by whoever walks past. There is no filter, so a semester-6 exam timetable sits next to a mechanical workshop. There is no confirmation that anyone saw an urgent circular, and once it is taken down it is gone. Parents and applicants cannot see admissions information without visiting.

For teachers and HODs: there is no way to address only one class, and approval is verbal, with no record. For the Principal: no control over who may post to whom, no history, and no idea how many people actually read an urgent notice.

So the problem we address is this: build one verified platform where a notice reaches exactly the audience its author chose, is approved by the right person first, and leaves an auditable record."

Slide 4 — Objectives · Speaker 2 · 1 minute

"Our objectives follow directly from that problem. One platform for five roles. Show every notice only to its intended audience, and enforce that on the server. Control who may address whom. Add quality control through an approval step for faculty notices. Admit only verified people through approved registration. Publish and expire on schedule. Track who has read and acknowledged. Keep attachments private. Serve visitors through a public website without exposing anything private, and keep an audit trail of every action."

Slide 5 — Existing System · Speaker 2 · 1 minute

"Today, a college relies on a physical board, sometimes a WhatsApp group. Both have the same weaknesses: the audience is not controlled, so anyone can post to everyone; there is no approval record; nothing tells the Principal who has read an urgent message; messages scroll away; and there is no public face for admissions beyond a static website that someone must update by hand."

Slide 6 — Proposed System · Speaker 2 · 1 minute 30 seconds

"Our system has two faces. A public website shows visitors the two wings, Engineering and Polytechnic, admissions information and public notices, with no sign-in. A portal for the five roles shows each person only the notices addressed to them.

Authors choose an audience: the whole college, a role, a department, a semester, a class, or specific people, in any combination, and they see how many people it will reach before publishing. Teachers' notices go to their HOD for approval. New accounts are approved by the right person: the class teacher, the HOD or the Principal."

Slide 7 — Technologies · Speaker 3 · 1 minute

"On the backend we use Java 17 and Spring Boot with Spring Security, Spring Data JPA and MySQL. Authentication uses signed JSON Web Tokens and BCrypt-hashed passwords. Attachments are stored in Cloudinary, email goes through Gmail SMTP, and an optional AI assistant uses the Anthropic API. The frontend is React with Vite, React Router, Axios and Bootstrap. We test with JUnit, Mockito and Spring's MockMvc on an in-memory database."

Slide 8 — System Architecture · Speaker 3 · 1 minute 30 seconds

"The system is three-tier. React runs in the browser, a Spring Boot API holds every rule, and MySQL stores the data, with Cloudinary for files. Inside the backend, a request passes through a security filter, a thin controller, a service layer where all business rules live, and repositories.

The key point is that authorisation has four independent checks: a valid token for an approved account; the right role for the endpoint; an allowed audience for that role; and permission for that specific notice. The browser is never trusted."

Slide 9 — Database Design · Speaker 3 · 1 minute

"There are twelve tables. Users, departments, notices and categories are the core. The idea that matters is the notice_targets table: each row says which role, department, semester, class or user a notice is for, and a notice is visible to a person if any row matches. Reads, acknowledgements and saved notices are unique per notice and user. The audit log has no foreign keys so history survives deletion."

Slide 10 — Major Modules I: accounts and approval · Speaker 4 · 1 minute 15 seconds

"Anyone can request an account as a student, teacher or HOD, but it starts pending and cannot sign in. The system chooses the approver: a student's class teacher, a teacher's HOD, an HOD's Principal, with fallbacks so no request is ever stuck. A rejected applicant sees the reason and can apply again.

Sign-in is protected: five failed attempts lock that email and address for fifteen minutes, and the error for an unknown email is identical to a wrong password, so the form cannot reveal who has an account."

Slide 11 — Major Modules II: notices · Speaker 4 · 1 minute 15 seconds

"Every notice starts as a draft. Principals, HODs and admins publish directly or schedule for later. Faculty submit to their HOD, who approves or rejects with a reason. A background job runs every minute to publish scheduled notices and expire old ones.

On publishing, the people addressed get an in-app notification, and an email if the notice is important or urgent. Each recipient can acknowledge, and the author sees how many of the intended people have read it. There is also an assistant that drafts a notice from one sentence, but it is always saved as a draft."

Slide 12 — Security and Reliability · Speaker 4 · 30 seconds

"Visibility is enforced by the backend on every read path, so changing an id in the URL returns forbidden. Uploads are checked by type, size and file signature, stored privately and downloaded through five-minute signed links. Suspending an account takes effect immediately even with a valid token. And we have seventy-two automated tests covering these rules."

Slide 13 — Live Demonstration · Speaker 5 · 6–7 minutes

"We will now show the system running. We will register a student, approve the account as the class teacher, write and submit a notice as a teacher, approve it as the HOD, and show that only the intended class can see it, including what happens when another student tries the direct link. Then we will publish an urgent college-wide notice as the Principal and look at the read statistics and the public site." ***Follow the Demo Script in §24.***

Slide 14 — Challenges Faced · Speaker 5 · 1 minute 15 seconds

"Our biggest lesson was that hiding things in the interface is not security, so we moved visibility into the backend and wrote the one matching rule twice, in Java and in SQL, with tests to keep them aligned. We also had to make an empty audience mean nobody, never everybody; keep Cloudinary and MySQL consistent when they cannot share a transaction; and upgrade databases that already contained data."

Slide 15 — Future Scope · Speaker 2 · 40 seconds

"Next we would add refresh tokens so sessions can be revoked, real-time push instead of polling, SMS and WhatsApp channels, a mobile app, notice templates, multilingual notices, bulk import of students, and Flyway migrations for production."

Slide 16 — Conclusion · Speaker 1 · 40 seconds

"To conclude: we replaced a physical board with a system where a notice reaches exactly the people it is for, passes through the right approval, and leaves a record of who has seen it, while a public website keeps visitors informed. Thank you. We are happy to take your questions."

§24 Complete Demo Script

24.1 Before you begin: a checklist

Done	Preparation
☐	MySQL is running (`docker compose up -d`). The schema is created by the backend.
☐	Backend is running from a **fresh** database so the seeder runs: `./mvnw spring-boot:run` (Windows: `mvnw.cmd spring-boot:run`).
☐	Frontend is running with `npm run dev`; open `http://localhost:5173`.
☐	Cloudinary variables are set if you want to show attachments; otherwise skip the attachment steps (uploads then return a clear error).
☐	Optional: `AI_API_KEY` set to show a real AI draft; otherwise the offline drafter is used, which also works.
☐	A second browser (or a private window) is ready, so two roles can be signed in at once.
☐	A small PDF is on the desktop. Browser zoom is about 90% so tables fit the projector.
☐	All seeded passwords are `Password@123`. The password for the account you register live must meet the rules.

24.2 The demo, step by step

#	What to click	What to say	Expected result
1	Open the home page, signed out. Scroll, then open **Public notices** and choose Engineering.	"This is the public site. Anyone can see the two wings and the notices marked public, with no account. Nothing private is here."	Landing page with both wings; public notice list filters.
2	Open any public notice. Change the number in the address bar to a private notice's id (e.g. 8).	"Private notices answer 'not found' here, exactly as if they did not exist."	404 / not-found page.
3	Login → **New here?** Register as Student: Engineering, Computer Engineering, semester 6, class A. First type a weak password.	"The checklist is live, and the same rule is enforced again on the server."	Checklist and strength bar; then 'Request sent… class teacher'.
4	Try to sign in as the new student.	"Correct password, but the account is pending, so no token. It tells the student who must approve."	Status card with a timeline.
5	Second browser: sign in as `faculty.cs@college.edu` → **Account requests**. Approve the new student.	"This teacher is the class teacher of CS 6A, so the system routed the request to them. The applicant never chooses."	Badge (count), request listed, approved toast.
6	As the same teacher: **Create notice**. In the assistant box type a sentence about a lab assignment; click *Write a draft*.	"This is saved as a draft, never published automatically. We always review."	Edit page with a drafted title, text, category, priority, dates.
7	Set audience **Class · Computer Engineering · semester 6 · class A**. Watch the count. Add a PDF. Click *Preview*, then *Save and submit for approval*.	"Before publishing I can see exactly how many people receive it. A teacher cannot publish; this goes to the HOD."	'N students will receive this notice'; status Pending approval.
8	Browser 1: sign in as `hod.cs@college.edu` → **Notice approvals** → open → **Approve**.	"Only the HOD of that department can approve. The same action publishes it and notifies the class."	Notice becomes Published; author notified.
9	Sign in as the newly approved student (or `student.cs6a@college.edu`). The notice appears. Open it and click **I have read this notice**.	"Students in 6A see it in their feed."	Notice visible; acknowledged.
10	Sign in as `student.cs6b@college.edu`. The notice is absent. Paste its URL.	"A different class. The list does not show it, and the server refuses a direct link."	403 'You are not authorized to view this notice'.
11	Back as the teacher or HOD: open the notice → **Show read statistics**, then scroll to the **History** panel.	"Counts are over the intended recipients only, and the history comes from the audit log."	Recipients, read, acknowledged; timeline of steps.
12	Sign in as `principal@college.edu`. Create an **Urgent** notice, audience **Entire college** (Private), and publish.	"The Principal publishes directly. Urgent notices are emailed too, and always require acknowledgement."	Published; red urgent banner on every member's dashboard.
13	As the Principal, create another notice and choose **Public**, *Show it under* **Engineering**; publish. Then refresh the signed-out site.	"Public notices are readable by visitors, but only while they are published, live and not expired."	Notice appears on the public home page and Public notices.

#	What to click	What to say	Expected result
14	Sign in as `admin@college.edu` → **Users** → suspend the new student with a reason. In the student's browser, click any link.	"Suspension takes effect immediately, even with a valid token, and the person is told why."	Student is signed out; sign-in shows the reason.
15	Admin → **Audit logs**. Then toggle dark mode from the top bar.	"Every action is recorded. And a quick look at the dark theme."	Filterable log; theme switches.

If something goes wrong on stage

- **Attachment error**: Cloudinary variables are probably not set. Say "attachments need cloud storage, which is optional" and continue without it.
- **No email**: mail is optional and disabled by default; the in-app notification is the main channel.
- **Account locked**: five wrong passwords lock that email for fifteen minutes. Use another seeded account.
- **Data already changed** from rehearsal: drop and recreate the database, or use another seeded student.

§25 Quick Revision Notes — 15-minute refresher

Everything essential in one place. Read this in the fifteen minutes before you present.

One-line pitch

A role-based college notice board — Spring Boot, React, MySQL — where each notice reaches exactly the audience its author chose, after the right approval, with a public website for visitors.

The five roles

- **Student**: self-registers; the class teacher approves (else HOD, then Principal)
- **Faculty (teacher)**: self-registers; the department HOD approves; notices need HOD approval
- **HOD**: self-registers; the Principal approves; one per department; publishes directly in own department
- **Principal**: created by Super Admin; publishes to anyone
- **Super Admin**: manages users, departments, categories, audit log

The killer answer: the matching rule

A `NoticeTarget` row matches a user when **every non-null field equals the user's value** (role, department, semester, class, user). A notice is visible if **any** row matches. Written in Java (`TargetMatcher`) and SQL (`targetedTo`), checked on every read path. Wrong id in the URL → **403**. Empty targets → **nobody**.

Eight notice statuses

`DRAFT` → `PUBLISHED` → `EXPIRED` → `ARCHIVED`. Future date → `SCHEDULED`. Faculty: `DRAFT` → `PENDING_APPROVAL` → (`APPROVED`) → `PUBLISHED`/`SCHEDULED`, or `REJECTED` → edit → `DRAFT`.
Scheduler: every minute; publishes due notices, expires old ones.

Auth in one breath

Rate-limit check → BCrypt `matches` (dummy hash if no user) → generic 401 on failure → status check only after a correct password (403 + `account`) → JWT (HS512, 24 h, `sub`=email, `uid`, `role`) → `localStorage` → Axios adds Bearer → filter reloads the user (blocks suspended) → `@PreAuthorize` → `TargetPolicy` → `NoticeAccessService`.

Attachments in one breath

Validate all (extension, MIME, ≤ 10 MB, magic bytes, ≤ 10 files) → upload to Cloudinary `authenticated` with a UUID name → row saved → on failure delete what was uploaded. Download → `canView` → 5-minute signed link. Delete → cloud first.

Public site rule

Visitors see a notice only if it is **public + published + live + not expired**. Anything else → **404**. Separate DTOs with no creator, targets, status or review data.

Numbers to remember

- **5** roles · **12** tables · **73** endpoints · **72** tests · **29** frontend routes
- **5** failed sign-ins per email+IP, **20** per IP, window **15 min**; **10** registrations per IP per hour
- JWT **24 h**; password-reset link **30 min**; download link **5 min**; scheduler **1 min**
- Files: pdf, doc, docx, jpg, jpeg, png · **10 MB** each · **10** per notice
- Semesters: B.Tech **8**, Diploma **6** · Password: **8–64** with upper, lower, digit, symbol
- Seed login: `admin@college.edu` / `Password@123` (also `principal@`, `hod.cs@`, `faculty.cs@`, `student.cs6a@`)

Know your limits (say them before the examiner does)

Logout does not revoke the token · rate limiter is in memory · admin can create two HODs · editing a published notice does not re-notify · stats load the recipient set in memory · `ddl-auto=update` is not for production.
For each: you know the fix (refresh tokens, Redis, call `ensureHodVacancy`, notify on change, SQL counts, Flyway).